



نظري

كلية الهندسة المعلوماتية

السنة الثانية

ملحق المحاضرات من 1
إلى 5

عمل طلابي



RB Informatcs; 09/09/2020

محتوى مجاني غير مخصص للبيع التجاري

بنیان الحاسوب ا

المحاضرة 1: المكونات البنيوية الرئيسية للحاسوب

س1: طالب يلاحظ أن الحاسوب لا يستطيع التمييز بين ما إذا كانت كلمة معينة مخزنة في الذاكرة تمثل تعليمة أم بيانة إلا من خلال السياق الذي يُقرأ به أثناء التنفيذ. أي مبدأ من مبادئ فون نيومان يفسر هذه الملاحظة بشكل مباشر؟

(أ) مبدأ الأداء عبر التنبؤ (Performance via Prediction)

(ب) مفهوم البرنامج المخزن (Stored Program Concept)

(ج) مبدأ التوافقية بين البرامج والمعالج

(د) مبدأ فصل النواقل السريع عن البطيء

الشرح: الفكرة الأساسية لمبدأ فون نيومان هي مفهوم البرنامج المخزن، أي يتم تخزين الأوامر في الذاكرة مثل البيانات. هذا التخزين المشترك هو تحدياً ما يجعل التمييز بين تعليمة وبيانة معتمداً على السياق (كيفية قراءة المعالج للكلمة) وليس على أي علامة فيزيائية مختلفة في الذاكرة.

الأخطاء: الخيار A يخلط بين مبدأ لاحق غير مرتبط مباشرة بهذه الملاحظة (التنبؤ يخص تسريع الأداء لا التمييز بين الأوامر والبيانات). الخيار C يقحم مفهوم التوافقية بين المعالجات، وهو مبدأ مختلف تماماً عن سبب عدم التمييز بين الأوامر والبيانات داخل الذاكرة. الخيار D يخلط بين تنظيم النواقل الفيزيائي وبين مبدأ فون نيومان الأساسي حول تخزين البرنامج.

الجواب: ب.

س2: أثناء تنفيذ برنامج تسلسلياً، يحتاج المعالج بعد جلب كل تعليمة أن يعرف عنوان التعليمة التالية التي سيجلبها دون تدخل يدوي من المبرمج في كل خطوة. أي سجل هو المسؤول تحديداً عن هذه المهمة، وكيف يؤديها؟



(أ) سجل عنوان الذاكرة (MAR)

(ب) سجل التعليم (IR)

(ج) سجل التحكم بالبرنامج (Program Control Register)

(د) المراكم (AC)

الشرح: Register Program Control هو المسؤول عن التنفيذ التسلسلي للبرامج، يقوم ب جلب أول تعليمية من الذاكرة لل CPU، ثم يقوم بعمل إزاحة بمقدار حجم التعليمية ليشير إلى التعليمية التالية.

الأخطاء: الخيار A يخلط بين وظيفة MAR العامة (حمل أي عنوان ذاكرة مطلوب الوصول إليه) والوظيفة التخصصية لسجل التحكم بالبرنامج في تتبع تسلسل التعليمات تحديداً. الخيار B يخلط بين IR (الذي يحمل محتوى التعليمية الحالية نفسها) والسجل الذي يحدد عنوان التعليمية التالية. الخيار D يقم AC، وهو سجل ينتمي لوحدة الحساب والمنطق ولا علاقة مباشرة له بتتبع عنوان التعليمية التالية.

الجواب: ج.

س3: يرسم طالب مخططاً لمعالج قديم يوضح فيه مجموعتين من السجلات: المجموعة الأولى (AC, MQ) والمجموعة الثانية (IR, IBR, MAR, PC)، ويحاول تصنيف كل مجموعة ضمن الوحدة الوظيفية الصحيحة داخل المعالج. إلى أي وحدة وظيفية تنتمي كل مجموعة من السجلات بحسب المخطط؟

(أ) AC و MQ ينتميان لوحدة الحساب والمنطق (ALU)، بينما IR و IBR و MAR و PC تنتمي لسجلات وحدة التحكم

(ب) جميع السجلات الستة تنتمي لوحدة التحكم حصراً

(ج) AC و MQ ينتميان لوحدة التحكم لأنهما يحفظان نتائج مؤقتة تحتاجها وحدة التحكم لاتخاذ القرار

(د) IR و IBR و MAR و PC تنتمي لوحدة الحساب والمنطق لأنها تشارك في تحديد عنوان العملية الحسابية القادمة

الشرح: سجلات وحدة التحكم هي PC-IR-IBR-MAR، وسجلات وحدة الحساب والمنطق هي MQ-AC-MBR. كما AC (المراكم) يُستخدم لحفظ النتائج المؤقتة ضمن وحدة الحساب والمنطق تحديداً.

الأخطاء: الخيار B يتجاهل التقسيم الصريح بين مجموعتي السجلات ويدمجها خطأً ضمن وحدة واحدة. الخيار C يعكس تصنيف AC و MQ بنسبتهما لوحدة التحكم بدلاً من وحدة الحساب والمنطق. الخيار D يعكس تصنيف سجلات وحدة التحكم بنسبتها خطأً لوحدة الحساب والمنطق.

الجواب: أ.

س4: مهندس يصمم معالجاً بحيث تكون جميع التعليمات موحدة الطول، وتُعالج بسرعة لأن حقول السجلات تقع في المكان نفسه ضمن كل تعليمة، ولا يُسمح بالوصول إلى الذاكرة إلا عبر تعليمتي تحميل وتخزين مخصصتين. أي فلسفة تصميم يعكسها هذا الوصف؟

(أ) CISC

(ب) RISC

(ج) التحكم البرمجي (Microprogrammed Control)

(د) مبدأ فون نيومان

الشرح: جميع التعليمات لها نفس الطول (عملية الجلب وفك الترميز بسيطة)، وحقول السجلات (Register Fields) هي المكان نفسه لجميع التعليمات، وتعليمة load تحمل من الذاكرة بينما تعليمة store تخزن فيها، وهذا التوصيف مطابق تماماً لما ورد في السؤال.

الأخطاء: الخيار A يعكس الفلسفة تماماً؛ فـCISC يعتمد على تعليمات معقدة متعددة الوظائف لا موحدة وبسيطة. الخيار C يخلط بين نوع وحدة التحكم (قرار تنظيمي منفصل غالباً ما يرتبط بـCISC لا RISC) وبين تصميم مجموعة التعليمات نفسها. الخيار D يقم مبدأ فون نيومان في سياق غير مرتبط به مباشرة؛ فتخزين البرنامج كبيانات لا يفرض بالضرورة توحيد طول التعليمات.

الجواب: ب.

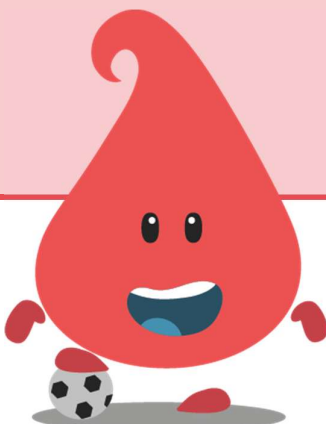
س5: عائلة معالجات تطورت تدريجياً عبر عقود بإضافة تعليمات معقدة أضيفت غالباً عبر البرمجيات أو الميكروكود قبل اللجوء لإضافة مكونات عتادية جديدة، وتتراوح أطوال تعليماتها بين 1 و17 بايت. أي نوع من التصميم تنتمي إليه هذه العائلة، وأي نوع من التحكم يغلب استخدامه معها؟

(أ) RISC مع تحكم عتادي (Hardwired)

(ب) RISC مع تحكم برمجي (Microprogrammed)

(ج) CISC مع تحكم عتادي (Hardwired)

(د) CISC مع تحكم برمجي (Microprogrammed)



الشرح: نظام Intel 80x86 و IBM 360/370 مجموعة تعليماتهم متغيرة الطول وتتراوح بين 1-17 بايت، وتصف هذه المعالجات ضمن فئة CISC، كما تنص على أن Microprogrammed Control يُستخدم غالباً في معالجات CISC.

الأخطاء: الخيار A و B يصنفان العائلة خطأ ضمن RISC، بينما وصف التطور التدريجي عبر الميكروكود وتغير طول التعليمات يطابق تحديداً وصف CISC (الذي يتميز بتعليمات متغيرة الطول، بعكس RISC الموحد الطول). الخيار C يصنف نوع العائلة (CISC) بشكل صحيح لكنه يخطئ في نوع التحكم؛ إذ يرتبط Hardwired Control بعائلات RISC غالباً، لا CISC.

الجواب: د.

س6: مهندسان يناقشان تصميم وحدة تحكم معالج جديد؛ الأول يقترح تصميمها بالكامل عبر دارات منطقية ثابتة، بينما يقترح الثاني تصميمها بحيث تعمل كبرنامج مصغر يولد إشارات التحكم. ما التصنيف الصحيح لكل من الاقتراحين، والفئة المعالجة التي يرتبط بها كل منهما غالباً؟

أ) الاقتراح الأول تحكم برمجي (Microprogrammed) مرتبط غالباً بـ RISC، والثاني تحكم عتادي (Hardwired) مرتبط غالباً بـ CISC

ب) الاقتراح الأول تحكم عتادي (Hardwired) يُستخدم غالباً في عائلات RISC، والثاني تحكم برمجي (Microprogrammed) يُستخدم غالباً في معالجات CISC

ج) كلا الاقتراحين يصفان النوع نفسه من التحكم

د) الاقتراح الأول يصف تصميم Datapath وليس وحدة تحكم، بينما الثاني وحده يصف وحدة تحكم فعلية

الشرح: Hardwired Control (عتادي) يتم تصميمه بشكل عتادي بالكامل ويُستخدم غالباً في عائلات RISC، و Microprogrammed Control (برمجي) تعمل فيه وحدة التحكم كبرنامج يولد إشارات التحكم ويُستخدم غالباً في معالجات CISC.

الأخطاء: الخيار A يعكس الارتباط الصحيح بين نوع التحكم وفئة المعالج مقارنة بما ورد. الخيار C ينكر وجود تمييز صريح بتعريفين منفصلين تماماً. الخيار D يخلط بين تصميم وحدة التحكم وتصميم مسار البيانات (Datapath)، وهما عنصران منفصلان تماماً في تصميم الحاسوب، والاقتراحان هنا يصفان وحدة تحكم فعلياً لا Datapath.

الجواب: ب.

س7: مهندس يلاحظ أن وضع وحدة ذاكرة صغيرة وسريعة بين المعالج والذاكرة الرئيسية (RAM) يرفع أداء الحاسوب بشكل ملحوظ رغم صغر سعتها مقارنة بالـ RAM. ما التفسير الأدق لهذا الأداء المحسّن؟

أ) الذاكرة الخبيئة تحل محل الـ RAM بالكامل في الأنظمة الحديثة

ب) الذاكرة الخبيئة تزيد من حجم الذاكرة الكلية القابلة للعبء من قبل المعالج

ج) لقرب الذاكرة الخبيئة من المعالج وحملها التعليمات المتكررة الطلب، فإنها تسرّع وصول المعالج إلى البيانات وتخفف الضغط عن الذاكرة الرئيسية

د) الذاكرة الخبيئة تحوّل البيانات من الصيغة الثنائية إلى صيغة يفهمها المستخدم مباشرة

الشرح: Cache Memory حل عتادي لرفع أداء الحاسوب (لقربها من الـ CPU وحملها للتعليمات) وهي وسيط بين المعالج والـ RAM، توضع بداخلها المعلومات التي يطلبها المعالج بشكل متكرر، حيث تنتقل المعطيات من الـ RAM إلى المعالج على شكل كتل Blocks.

الأخطاء: الخيار A يتجاوز الوصف؛ فالذاكرة الخبيئة وسيط بين المعالج والـ RAM وليست بديلاً كاملاً عنها. الخيار B يخلط بين وظيفة الذاكرة الخبيئة (تسريع الوصول) ووظيفة مختلفة تماماً (زيادة السعة الكلية)، غير مذكورة كوظيفة للكاش. الخيار D يقدم وظيفة تحويل الصيغة في سياق غير متعلق بها.

الجواب: ج.



كُنِ الْبَطْلَ الَّذِي تُحِبُّ ...

فَالْقِصَّةُ قِصَّتُكَ!!

س8: نظام تشغيل يسمح لبرنامج بالعمل وكأن لديه مساحة ذاكرة أكبر بكثير من السعة الفعلية لذاكرة RAM المتوفرة فيزيائياً على الجهاز. أي حل تقني من الحلول يفسر هذه الإمكانية؟

- (أ) الذاكرة الخبيثة (Cache Memory)
- (ب) الذاكرة المتطايرة (RAM)
- (ج) مبدأ التوافقية
- (د) الذاكرة الافتراضية (Virtual Memory)

الشرح: Virtual Memory حل تقني يقوم نظام التشغيل بعنونة جزء من القرص الصلب ويعتبره امتداداً للذاكرة RAM، وهذا يفسر مباشرة كيف يمكن لبرنامج العمل بمساحة ذاكرة منطقية أكبر من سعة الـ RAM الفعلية.

الأخطاء: الخيار A يخلط بين وظيفتين متعاكستين تقريباً؛ الكاش يسرّع الوصول لبيانات موجودة أصلاً في RAM، بينما المطلوب هنا توسيع السعة المتاحة لا تسريع الوصول. الخيار B يخترع خاصية غير واقعية وغير مذكورة. الخيار C يقدم مفهوم التوافقية في سياق لا علاقة له بتوسيع سعة الذاكرة.

الجواب: د.

س9: في تصميم Typical System Architecture المحسّن، يوجد طريق سريع مخصص للمعالج والذاكرة، وطريق فرعي أبطأ مخصص لأجهزة الإدخال والإخراج (Bus O/I)، ومكوّن يربط بين الطريقتين. ما وظيفة هذا المكوّن الرابط (Bus Adapter)؟

- (أ) رفع تردد الساعة الخاص بالمعالج ليتوافق مع سرعة أجهزة الإدخال والإخراج البطيئة
- (ب) قطعة تصل بين الطريق السريع والطريق البطيء لتنظيم انتقال البيانات بينهما
- (ج) تخزين التعليمات المتكررة الطلب بشكل مؤقت، بنفس وظيفة الذاكرة الخبيثة تماماً
- (د) تحويل الإشارات الرقمية داخل المعالج إلى إشارات تماثلية تفهمها الطرفيات مباشرة

الشرح: جسر الربط (Bus Adapter) قطعة تصل بين طريقتين السريع والبطيء لتنظيم البيانات، وذلك ضمن سياق فصل الطرق (Buses) لتقليل الازدحام.

الأخطاء: الخيار A يخترع وظيفة غير واردة؛ فالحل المعتمد هو فصل الطرق وربطها عبر جسر، لا تعديل تردد المعالج نفسه. الخيار C يخلط بين وظيفة جسر الربط ووظيفة الذاكرة الخبيثة، وهما مكونان منفصلان تماماً في المخطط رغم أن كليهما يهدف لتحسين الأداء. الخيار D يقحم وظيفة تخص وحدات إدخال/إخراج معينة في سياق غير متعلق بها.

الجواب: ب.

س10: عندما يريد المعالج قراءة قيمة من الذاكرة، يجب أولاً تحديد موقعها بالضبط (رقم الخانة)، ثم تحديد نوع العملية المطلوبة (قراءة أم كتابة)، ثم تنتقل القيمة الفعلية بين الذاكرة والمعالج. ما الأنواع الثلاثة لنواقل النقل المسؤولة عن هذه العمليات الثلاث؟

(أ) ناقل الجلب، ناقل فك الترميز، ناقل التنفيذ

(ب) ناقل القراءة، ناقل الكتابة، ناقل الانتظار

(ج) ناقل البيانات (Data)، ناقل العنوان (Address)، ناقل التحكم (Control)

(د) الناقل السريع، الناقل البطيء، جسر الربط بينهما

الشرح: النواقل (Buses) وحدات مسؤولة عن نقل البيانات، تربط بين المعالج والذاكرة ووحدات الإدخال والإخراج، ولتنقل ثلاثة أنواع: بيانات (Data)، عنوان (Address)، تحكم (Control).

الأخطاء: الخيار A يخلط بين مراحل تنفيذ التعليمات (خطوات زمنية داخل المعالج) وبين أنواع النواقل الفيزيائية (مسارات نقل مادية)، وهما مفهومان مختلفان تماماً. الخيار B يخترع تصنيفاً غير وارد؛ فالقراءة والكتابة نوعا عملية ينقلهما ناقل التحكم، لا ناقلان منفصلان. الخيار D يخلط بين تصنيف النواقل حسب نوع المحتوى المنقول وتصنيفها حسب السرعة الفيزيائية، وهو سياق مختلف تماماً.

الجواب: ج.

You are The ARCHITECT of the future software

س11: خط توارد (Pipeline) مكوّن من 5 مراحل يُعالج تدفقاً من 5 تعليمات فقط، أي أن عدد التعليمات مساوٍ لعدد المراحل. بحسب الوصف عن تقنية التوارد، ماذا يحدث خلال الفترة الزمنية اللازمة لملء خط التوارد بهذه التعليمات الخمس؟

- (أ) تكتمل جميع التعليمات الخمس في الدورة نفسها بفضل التنفيذ المتوازي الكامل
 (ب) لا تكتمل أي تعليمة إطلاقاً حتى تنتهي جميع التعليمات الخمس من مرحلة الجلب أولاً
 (ج) يتوقف خط التوارد بالكامل عن العمل لأن عدد التعليمات يساوي عدد المراحل بالضبط
 (د) لا تكتمل سوى تعليمة واحدة فقط خلال هذه الفترة

الشرح: بفرض أن كل التعليمات التي سيتم تنفيذها تساوي عدد المراحل، فلا يتم تنفيذ سوى تعليمة واحدة، أي أن ملء خط التوارد لأول مرة يستغرق وقتاً لا يكتمل خلاله سوى تعليمة واحدة فقط رغم دخول التعليمات الأخرى إلى مراحل مختلفة من الخط.

الأخطاء: الخيار A يناقض جوهر فكرة التوارد؛ فالتداخل بين مراحل تعليمات مختلفة لا يعني اكتمال تنفيذها جميعاً في اللحظة نفسها. الخيار B يتجاهل مبدأ التداخل (Overlap) الذي يميز التوارد أصلاً عن التنفيذ التسلسلي البحت. الخيار C يخترع نتيجة غير واردة، التي لا تذكر أي توقف بل تصف استمرار العمل مع اكتمال تعليمة واحدة فقط.

الجواب: د.

س12: طالب يحاول ترتيب مراحل تنفيذ خط التوارد (Pipeline) الخمس بالترتيب الصحيح. ما الترتيب الصحيح لهذه المراحل؟

- (أ) فك ترميز التعليمة (ID) ← جلب التعليمة (IF) ← تنفيذ التعليمة (Execute) ← جلب المعاملات (OF) ← الكتابة في الذاكرة (WB)
 (ب) جلب التعليمة (IF) ← جلب المعاملات (OF) ← فك ترميز التعليمة (ID) ← تنفيذ التعليمة (Execute) ← الكتابة في الذاكرة (WB)
 (ج) جلب التعليمة (IF) ← فك ترميز التعليمة (ID) ← جلب المعاملات (OF) ← تنفيذ التعليمة (Execute) ← الكتابة في الذاكرة (WB)
 (د) جلب التعليمة (IF) ← فك ترميز التعليمة (ID) ← تنفيذ التعليمة (Execute) ← جلب المعاملات (OF) ← الكتابة في الذاكرة (WB)

الشرح: مراحل تنفيذ التوارد بالترتيب الصحيح هي: 1. جلب التعليمية 2. فك (Instruction Fetch). فك ترميز التعليمية 3. (ID). جلب المعاملات 4. (OF). تنفيذ التعليمية 5. (Execute). الكتابة في الذاكرة (WB).

الأخطاء: الخيار A يقدم مرحلة فك الترميز على مرحلة الجلب، وهو ترتيب غير منطقي أصلاً ومخالف للنص. الخيار B يبدل ترتيب مرحلتي جلب المعاملات وفك الترميز مقارنة بالنص الأصلي. الخيار D يبدل ترتيب مرحلتي التنفيذ وجلب المعاملات، بينما يجب جلب المعاملات أولاً قبل تنفيذ العملية عليها.

الجواب: ج.

س13: يقرأ طالب مقالاً حديثاً يزعم كاتبه أن قانون مور انتهى بشكل كامل ونهائي ولم يعد له أي أثر على صناعة المعالجات. ما التقييم الأدق لهذا الزعم بحسب ما ورد؟

- (أ) الزعم غير دقيق؛ إذ يُذكر أن قانون مور تباطأ جزئياً لكنه ما يزال مستمراً، وليس متوقفاً بشكل كامل ونهائي
(ب) الزعم دقيق تماماً؛ بينما الصحيح أن قانون مور توقف نهائياً ولم يعد قابلاً للتطبيق إطلاقاً
(ج) الزعم غير قابل للتقييم؛ لأن هذا لم يُذكر إطلاقاً لموضوع قانون مور أو حالته الراهنة
(د) الزعم دقيق جزئياً فقط فيما يخص المعالجات المخصصة للهواتف الذكية دون غيرها من الشرائح

الشرح: كل سنة ونصف إلى سنتين يزداد عدد الترانزستورات الموجودة على الشريحة الواحدة بمقدار الضعف، أما حالياً تباطأ جزئياً ولكنه مستمر.



الأخطاء: الخيار B يناقض هذا مباشرة بادعاء التوقف الكامل والنهائي. الخيار C يتجاهل التخصيص الصريح لفقرة كاملة لقانون مور وحالته الراهنة. الخيار D يخترع تخصيصاً بفئة أجهزة معينة غير مذكور إطلاقاً.

الجواب: أ.

س14: في منتصف الألفينات، توقف تصغير الترانزستورات عن خفض استهلاك الطاقة بالنسبة نفسها، وبدأت الحرارة تزداد رغم استمرار التصغير الفيزيائي. كيف تعامل مصنّعو المعالجات مع هذا التحدي؟

- (أ) بالتخلي الكامل عن تقنية الدارات المتكاملة (IC) والعودة إلى الترانزستورات المنفصلة لتقليل الحرارة
- (ب) باستخدام أنوية متعددة داخل المعالج (بعضها قوي وبعضها موفر للطاقة)، والتركيز على تحسين الكفاءة بدلاً من رفع التردد فقط، عبر تقنيات المعالجة المتوازية
- (ج) بإزالة الذاكرة الخبيئة بالكامل من تصميم المعالج لتقليل عدد الترانزستورات وبالتالي الحرارة الناتجة
- (د) باستبدال الذاكرة الافتراضية بذاكرة RAM أكبر حجماً لتعويض فقدان الأداء الناتج عن ارتفاع الحرارة

الشرح: لجؤوا إلى زيادة السرعة عن طريق استخدام أنوية متعددة، بعضها قوية وبعضها موفر للطاقة، والعمل على تحسين الكفاءة بدل السرعة (زيادة التردد)، وطوروا تقنيات المعالجة المتوازية.

الأخطاء: الخيار A يخترع حلاً غير واقعي وغير مذكور إطلاقاً. الخيار C يقم بإزالة الذاكرة الخبيئة، وهو حل يتعارض مع الاتجاه الفعلي لتطوير المعالجات. الخيار D يخلط بين حل تقني غير متعلق بالمشكلة أصلاً؛ فالذاكرة الافتراضية تخص سعة التخزين المنطقية لا استهلاك الطاقة.

الجواب: ب.

س15: يقارن مهندس بين نظامين حاسوبيين لتنفيذ نفس نوع المهام: النظام A ينهي تنفيذ مهمة واحدة بمفرده بشكل أسرع من النظام B، لكن عند تشغيل عدد كبير من المهام دفعة واحدة، ينجز النظام B عدداً أكبر من المهام الكلية خلال نفس الفترة الزمنية. أي من مقياسي الأداء يُظهر كل نظام تفوقه فيه؟

- (أ) النظام A يتفوق في زمن الاستجابة (Time Response)، بينما النظام B يتفوق في الإنتاجية (Throughput)
- (ب) النظام A يتفوق في الإنتاجية (Throughput)، بينما النظام B يتفوق في زمن الاستجابة (Time Response)
- (ج) كلا النظامين متكافئان تماماً في كلا المقياسين
- (د) لا يمكن تحديد أي نظام أفضل رغم أن هناك تعريفين منفصلين ومختلفين تماماً لـ زمن الاستجابة أو الإنتاجية كمقياسين منفصلين للأداء

الشرح: يُعرّف زمن الاستجابة (Time Response) بأنه زمن التنفيذ أو الوقت المستغرق حتى يصبح الأمر في حالة تنفيذ، وتُعرّف الإنتاجية (Throughput) بأنها العمل المنجز خلال واحدة الزمن. إنهاء مهمة مفردة أسرع = تفوق في زمن الاستجابة، وإنجاز عدد أكبر من المهام الكلية خلال نفس الوقت = تفوق في الإنتاجية.

الأخطاء: الخيار B يعكس تصنيف المقياسين بالكامل مقارنة بمعطيات السيناريو. الخيار C يتجاهل كون المقياسين تعريفيين منفصلين ومختلفين تماماً. الخيار D يناقض هذا التي تخصص فقرة صريحة لتعريف أهم مقياسين يستخدمان لتقييم أداء الحاسوب.

الجواب: أ.

س16: يقرر مهندس أول أن المعالج الجديد يجب أن يدعم تعليمة ضرب (multiply) ضمن مجموعة تعليماته، بينما يقرر مهندس ثانٍ لاحقاً الطريقة الفعلية لتنفيذ هذه التعليمة: عبر دائرة ضارب (multiplier) مخصصة أم عبر دائرة جامع (adder) تُكرر عملية الجمع. أي القرارين يندرج تحت Architecture وأيها تحت Organization بحسب المثال؟

(أ) كلا القرارين يندرجان تحت Organization لأنهما يخطان تنفيذ عملية حسابية واحدة
(ب) القرار الأول (دعم التعليمة) هو Organization، والقرار الثاني (طريقة التنفيذ) هو Architecture
(ج) لا يمكن التمييز بينهما رغم وجود تمييز واضح بين Organization و Architecture كمترادفين
(د) القرار الأول (دعم تعليمة الضرب ضمن مجموعة التعليمات) هو قرار Architecture، والقرار الثاني (تنفيذها عبر دائرة ضارب أو جامع) هو قرار Organization

الشرح: هذا المثال يوضح ذلك: كتابة إحدى تعليمات الجمع أو الطرح أو الضرب هي مسألة بنيوية (Architecture)، فمثلاً كتابة تعليمة الضرب هي مسألة بنيوية، أما تحقيق هذه العملية عن طريق ضارب أو دائرة الجامع فهي مسألة تنظيمية (Organization).

الأخطاء: الخيار A يتجاهل التمييز الأساسي بين ما هو ظاهر للمبرمج (دعم التعليمة) وما هو خفي عنه (طريقة التنفيذ الفعلية). الخيار B يعكس التصنيف الصحيح بالكامل مقارنة بالتصنيف الصحيح. الخيار C ينفي وجود أي فرق بين المفهومين رغم تخصيص جدولاً ومثالاً كاملين للتفريق بينهما.

الجواب: د.

س17: مهندسان يناقشان قرارى تصميم لنظام حاسوب جديد: الأول يحدد عدد البتات اللازمة لتمثيل نوع بيانات معين ضمن مجموعة التعليمات، والثاني يحدد تقنية الذاكرة المستخدمة فعلياً (مثل نوع DRAM) وآلية توليد إشارات التحكم الخاصة بها. أي القرارين يندرج تحت Organization؟

(أ) القرار الأول فقط (عدد البتات لتمثيل نوع البيانات)

(ب) كلا القرارين يندرجان تحت Architecture

(ج) القرار الثاني فقط (تقنية الذاكرة المستخدمة وآلية إشارات التحكم)

(د) لا ينتمي أي منهما إلى Architecture أو Organization، بل إلى مجال منفصل تماماً يسمى فقط Implementation

الشرح: Organization تشمل تفاصيل العتاديات (Hardware) التي تكون شفافة بالنسبة للمبرمج، مثل إشارة التحكم وتقانة الذاكرة المستخدمة، بينما تصنف Architecture عناصر مثل عدد البتات اللازمة لتنفيذ المعطيات ضمن ما هو ظاهر للمبرمج.

الأخطاء: الخيار A يعكس التصنيف الصحيح تماماً؛ فعدد البتات لتمثيل نوع البيانات هو قرار Architecture ظاهر للمبرمج، لا Organization. الخيار B يصنف كلا القرارين خطأً ضمن Architecture، متجاهلاً أن تقنية الذاكرة الفعلية وإشارات التحكم أمثلة صريحة على Organization. الخيار D يخترع تصنيفاً ثالثاً غير وارد إطلاقاً.

الجواب: ج.

س18: تطرح شركة معالجات نموذجاً جديداً ضمن نفس عائلة معالجاتها القديمة، بحيث يحافظ النموذج الجديد على تشغيل البرامج القديمة نفسها دون أي تعديل، لكنه يحقق أداءً أعلى بفضل تغييرات داخلية في طريقة تنفيذ العمليات. أي وصف يعكس بدقة ما يتغير وما يبقى ثابتاً بين النموذجين وفق الجدول المقارن؟

(أ) تصميم المعالج ومعماريته (Architecture) يبقيان ثابتين ضمن العائلة الواحدة، بينما التنظيم (Organization) هو غير الثابت الذي يتغير بين النماذج

(ب) كل من Architecture و Organization يجب أن يتغيرا معاً في كل نموذج جديد، وإلا فلن يكون هناك أي تحسين حقيقي في الأداء

(ج) Architecture هو غير الثابت الذي يتغير بين النماذج، بينما Organization يبقى ثابتاً دائماً ضمن العائلة الواحدة

(د) لا علاقة بين ثبات أي من المفهومين وبين انتماء المعالج لعائلة معينة، فهذا مفهوم غير مرتبط بالموضوع

الشرح: يوضح الجدول المقارن أن تصميم الحاسوب ومعماريته ثابتة ضمن العائلة الواحدة (خانة Architecture)، بينما التنظيم (Organization) غير ثابت ضمن العائلة الواحدة. هذا يفسر بقاء التوافقية البرمجية رغم تحسّن الأداء الداخلي.

الأخطاء: الخيار B يناقض المبدأ الأساسي وراء التوافقية بين أجيال العائلة الواحدة، والذي يعتمد أصلاً على ثبات Architecture. الخيار C يعكس التصنيف الصحيح تماماً كما ورد في الجدول المقارن. الخيار D ينكر وجود هذا التمييز رغم تخصيص له سطرًا صريحاً ضمن جدول المقارنة.

الجواب: أ.

س19: بعد اعتماد شركة تصنيع معالجات لمجموعة تعليمات (ISA) معينة ونشرها في السوق لسنوات طويلة، تكتشف الشركة حاجة لتعديل جوهري في بعض التعليمات لتحسين الأداء. ما التحدي الرئيسي الذي ستواجهه الشركة؟

- (أ) يصبح من الصعب تغيير التعليمات المُثبتة مسبقاً أو الإضافة عليها، وهذا من أبرز مساوئ ISA
- (ب) لن تواجه الشركة أي تحدٍ
- (ج) ستحتاج الشركة لإعادة تصميم الذاكرة الخبيئة بالكامل قبل أي تعديل على ISA لأنها مرتبطةان حصراً
- (د) ستفقد الشركة تلقائياً حق استخدام اسمها التجاري لعائلة معالجاتها عند أي تعديل على مجموعة التعليمات

الشرح: بعد تثبيت التعليمات يصبح من الصعب تغييرها أو الإضافة عليها.

الأخطاء: الخيار B يناقض هذا مباشرة التي تذكر هذه النقطة كمساوئ صريح لISA. الخيار C يخترع ارتباطاً حصرياً غير موجود بين تعديل ISA وإعادة تصميم الذاكرة الخبيئة بالكامل. الخيار D يخترع عقوبة تجارية غير مذكورة إطلاقاً في سياق مساوئ ISA.

الجواب: أ.

س20: يحاول طالب أن يفهم لماذا يستطيع نفس البرنامج المُصرّف (compiled) العمل على معالجات مختلفة فيزيائياً طالما أنها تنتمي لنفس مجموعة التعليمات، رغم اختلاف التصميم الداخلي (Datapath) والدارات المنطقية) بين هذه المعالجات. أي تفسير يعكس بدقة موقع ودور ISA ضمن طبقات النظام الحاسوبي؟

(أ) ISA يقع كحد فاصل بين طبقة البرمجيات (نظام التشغيل والمترجم) من فوق وطبقة العتاد (Datapath والتحكم والدوائر المنطقية) من تحت، وهذا الموقع الوسيط هو ما يسمح بتوحيد واجهة البرمجة رغم اختلاف التنفيذ الداخلي للعتاد

(ب) ISA هو جزء من طبقة العتاد السفلى مباشرة (أي هو نفسه تصميم Datapath والدوائر المنطقية)، ولهذا السبب يعمل البرنامج نفسه على معالجات مختلفة

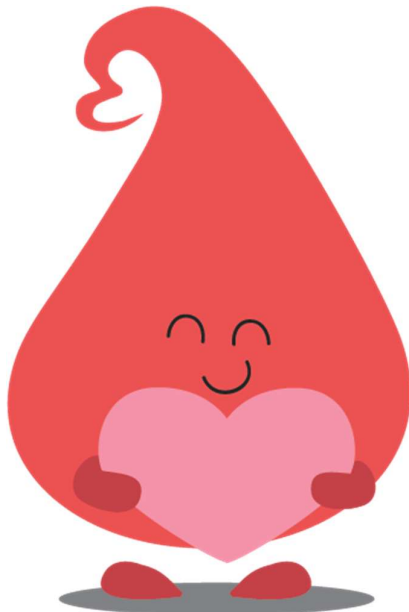
(ج) ISA يقع فوق طبقة نظام التشغيل والمترجم مباشرة، وهو المسؤول عن كتابة واجهات المستخدم للبرامج التطبيقية

(د) لا يوجد أي تسلسل طبقات في تصميم الحاسوب، بل يُعتبر العتاد والبرمجيات وحدة واحدة غير قابلة للفصل

الشرح: ISA يُستخدم لتبسيط التصميم عبر التجريد (abstract)، ويوضع كحد فاصل بين العتاد والبرمجيات، وهذا هو بالضبط ما يفسر إمكانية تشغيل البرنامج نفسه على معالجات مختلفة الداخل طالما أنها تشترك في نفس ISA من الخارج.

الأخطاء: الخيار B يخلط بين ISA (طبقة تجريد وسيطة) و Datapath (التنفيذ الفعلي للعتاد)، وهما طبقتان منفصلتان تماماً، بل إن اختلافهما هو ما يفسر إمكانية التوافق البرمجي. الخيار C يضع ISA في موقع خاطئ تماماً، بينما هو في الحقيقة طبقة وسطى تحت طبقة نظام التشغيل. الخيار D ينكر وجود أي تسلسل طبقي رغم أن هناك وصفاً صريحاً لفكرة طبقة عليا وطبقة وسطى وطبقة سفلى.

الجواب: أ.



Make sure your score is full on ones
EXCEPT the MSB which is for sign 😊

المحاضرة 2: طبقات التجريد وقياس الأداء

س1: يعمل برنامج معين لمدة 10 ثوانٍ على الحاسوب A الذي يملك تردد ساعة يساوي 2 جيجا هرتز. يريد مصمم حواسيب بناء حاسوب جديد B يُشغّل البرنامج نفسه خلال 6 ثوانٍ فقط، لكن التصميم الجديد سيتطلب من الحاسوب B تنفيذ 1.2 ضعف عدد دورات الساعة التي يحتاجها الحاسوب A لتنفيذ البرنامج نفسه. ما تردد الساعة الذي يجب أن يستخدمه الحاسوب B لتحقيق هذا الهدف؟

(أ) 4 جيجا هرتز

(ب) 3.33 جيجا هرتز

(ج) 2.4 جيجا هرتز

(د) 4.8 جيجا هرتز

الشرح: دورات A = تردد A × زمن A = $2 \times 10^9 \times 10 = 20 \times 10^9$ دورة. دورات B = 1.2 × دورات A = 24×10^9 دورة. تردد B = دورات B ÷ زمن B = $24 \times 10^9 \div 6 = 4 \times 10^9$ هرتز = 4 جيجا هرتز، وهذا مطابق تماماً للحل.

الأخطاء: تجاهل عامل 1.2 يعني معاملة عدد دورات B وكأنه مساوٍ لعدد دورات A، وهو افتراض يخالف معطيات المسألة صراحة. استخدام 10 ثوانٍ بدلاً من 6 ثوانٍ عند حساب تردد B يخلط بين زمن تنفيذ الحاسوب القديم والحاسوب الجديد المستهدف. ضرب الناتج النهائي بـ 1.2 مرة إضافية يكرر تطبيق العامل مرتين، بينما يجب تطبيقه مرة واحدة فقط على عدد الدورات.

الجواب: أ.

س2: حاسوبان A و B يستخدمان نفس مجموعة التعليمات لتنفيذ البرنامج نفسه. الحاسوب A: زمن دورة الساعة 250 ps ومتوسط CPI يساوي 2.0. الحاسوب B: زمن دورة الساعة 500 ps ومتوسط CPI يساوي 1.2. أي الحاسوبين أسرع في تنفيذ هذا البرنامج، وبأي مقدار؟

(أ) الحاسوب B أسرع من A بمقدار 1.2 مرة

(ب) الحاسوب A أسرع من B بمقدار 1.2 مرة

(ج) الحاسوبان متساويان في الأداء

(د) لا يمكن تحديد الأسرع دون معرفة عدد التعليمات (C) بالضبط

الشرح: CPU Time(A) = CPI(A) × IC × ClockTime(A) = 2.0 × IC × 250 = IC×500ps. CPU Time(B) = 1.2 × IC × 500 = IC×600ps. بما أن IC نفسه في الحاسوبين (نفس ISA)، فإن 600 > 500، أي CPU Time(B) > CPU Time(A)، فالحاسوب A أسرع، ومقدار التفوق 1.2 = 500/600 مرة.

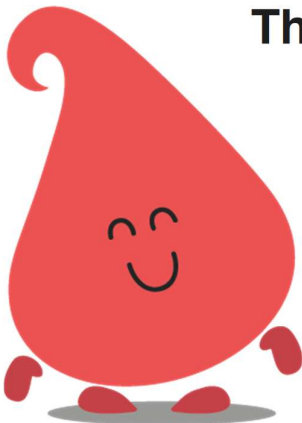
الأخطاء: الحكم على السرعة بالاعتماد على CPI فقط دون مراعاة زمن دورة الساعة هو تحديداً خطأ شائع يجب الحذر منه: أداء المعالج لا يعتمد على عامل واحد بل هو نتاج عوامل مترابطة. افتراض تقارب الأداء دون إجراء الحساب الفعلي يتجاهل الفارق الواضح بين 500 و 600 بيكو ثانية. الادعاء بالحاجة لمعرفة IC يتجاهل أن عدد التعليمات متطابق أصلاً في الحاسوبين (للاستخدامهما نفس مجموعة التعليمات)، وبالتالي يُختصر تلقائياً من النسبة.

الجواب: ب.

س3: حاسوبان A و B ينفذان البرنامج نفسه بالمواصفات التالية: (A) عدد التعليمات 10 مليار، تردد الساعة 4.2 جيجا هرتز، (B) CPI=1.0، عدد التعليمات 8 مليار، تردد الساعة 4.0 جيجا هرتز، CPI=1.1. بحساب معدل MIPS وزمن التنفيذ الفعلي لكل حاسوب، أي استنتاج هو الصحيح؟

- (أ) الحاسوب A يملك MIPS أعلى (4200 مقابل 3636 للحاسوب B) ولذلك فهو الأسرع فعلياً في زمن التنفيذ أيضاً
- (ب) الحاسوب B يملك MIPS أعلى وهو أيضاً الأسرع فعلياً في زمن التنفيذ
- (ج) الحاسوب A يملك MIPS أعلى (4200 مقابل 3636)، لكن زمن التنفيذ الفعلي يظهر أن B أسرع فعلياً (2.20 ثانية مقابل 2.38 ثانية لـ A)، بسبب اختلاف عدد التعليمات الكلي بين الجهازين
- (د) قيمة MIPS ترتبط دائماً طردياً مع زمن التنفيذ الفعلي، لذلك التيجتان يجب أن تتفقا بالضرورة

Think more before you see the answer!! ↩



الشرح: MIPS(A) = $4.2 \times 10^9 / (1.0 \times 10^6) = 4200$. MIPS(B) = $4.0 \times 10^9 / (1.1 \times 10^6) = 3636$ أعلى. لكن $ExecutionTime(A) = (10 \times 10^9 \times 1.0) / 4.2 \times 10^9 = 2.38s$ و $ExecutionTime(B) = (8 \times 10^9 \times 1.1) / 4.0 \times 10^9 = 2.20s$ أي B أسرع فعلياً. السبب: A احتاج تنفيذ مليار تعليمة إضافية عن B، وهذا بالضبط الدرس المستفاد من هذا المثال: MIPS يمكن أن يتغير بعلاقة عكسية مع الأداء الفعلي.

الأخطاء: الافتراض بأن MIPS الأعلى يعني بالضرورة أداءً أسرع فعلياً هو تحديداً الفخ الذي يحذر منه هذا المثال. عكس تصنيف أي حاسوب يملك MIPS الأعلى يناقض الحساب الفعلي (A هو صاحب القيمة الأعلى، لا B). افتراض علاقة طردية ثابتة بين MIPS وزمن التنفيذ الفعلي يناقض الاستنتاج الصريح المبني على هذا المثال نفسه.

الجواب: ج.

4: قياسات على معالج معين تُظهر أن نسبة تكرار جميع تعليمات الفاصلة العائمة FP مع عملية الجذر التربيعي SQRT مجتمعين تساوي 30% بمتوسط CPI يساوي 4، بينما نسبة تكرار عمليات SQRT وحدها تساوي 2% بمتوسط CPI يساوي 10. ما قيمة CPI الخاصة بتعليمات FP فقط (باستثناء SQRT)؟

أ) 4.00

ب) 2.00

ج) 10.00

د) 3.57

الشرح: نسبة تكرار FP وحدها = $30\% - 2\% = 28\%$. من معادلة CPI الموزون:
 $averageCPI(FP+SQRT) = (CPI(FP) \times IC(FP) + CPI(SQRT) \times IC(SQRT)) \div IC(FP+SQRT)$
 بالتعويض: $4 = (CPI(FP) \times 0.28 + 10 \times 0.02) \div 0.30 \rightarrow 1.2 = CPI(FP) \times 0.28 + 0.2 \rightarrow CPI(FP) = 3.57$
 مطابق تماماً للحل الصحيح.

الأخطاء: استخدام القيمة 4 مباشرة يتجاهل أنها تمثل متوسط الفئتين مجتمعين لا فئة FP وحدها. طرح قيم CPI من بعضها مباشرة (4-10) خطأ إجرائي يتجاهل أن CPI الموزون يُحسب بالنسب لا بالطرح المباشر بين القيم. افتراض تساوي CPI بين الفئتين يناقض المعطيات الصريحة في المسألة التي تحدد قيمتين مختلفتين تماماً (4 و 10).

الجواب: د.



س5: نَقِّذ الحاسوب A برنامجاً معيناً خلال 10 ثوانٍ، بينما استغرق الحاسوب B تنفيذ البرنامج نفسه 15 ثانية. بحسب معادلة الأداء النسبي $n = \text{ExecutionTime}(B)/\text{ExecutionTime}(A)$ ، ما مقدار تفوق الحاسوب A على الحاسوب B؟

- (أ) الحاسوب A أسرع من B بمقدار 1.5 مرة ($1.5 = 10/15$)
 (ب) الحاسوب A أسرع من B بمقدار 0.67 مرة ($0.67 = 15/10$)
 (ج) الحاسوبان متساويان في الأداء لأن الفرق بينهما 5 ثوانٍ فقط
 (د) الحاسوب B أسرع من A بمقدار 1.5 مرة لأنه ذو ترقيم لاحق يوجي بتطور أكبر

الشرح: بتطبيق القانون مباشرة: $n = \text{ExecutionTime}(B)/\text{ExecutionTime}(A) = 15s/10s = 1.5$ أي أن الحاسوب A أسرع من B بمقدار 1.5 مرة، تماماً كما في المثال المحلول.

الأخطاء: قسمة زمن A على زمن B بدلاً من العكس تعكس اتجاه النسبة المحددة في القانون، وتنتج قيمة أقل من 1 رغم أن A هو الأسرع فعلياً (فالقيمة يجب أن تكون أكبر من 1 عندما يكون البسط زمن الحاسوب الأبطأ). اعتبار الفارق الزمني المطلق (5 ثوانٍ) مقياساً للتكافؤ يتجاهل أن المعادلة تعتمد على النسبة لا الفارق. الاستناد إلى ترقيم الأجهزة كمعيار للسرعة معيار غير منطقي وغير مذكور إطلاقاً.

الجواب: أ.

س6: يقيس مهندس الزمن الكلي من لحظة طلب المستخدم لتشغيل برنامج معين وحتى ظهور النتيجة على الشاشة، ويلاحظ أن هذا الزمن أكبر بكثير من الزمن الذي قضاها المعالج فعلياً في تنفيذ تعليمات البرنامج. أي تفسير يعكس بدقة الفرق بين الزمن المنقضي (Elapsed Time) وزمن المعالج (CPU Time)؟

- (أ) الزمن المنقضي يساوي دائماً زمن المعالج تماماً، والفارق الملحوظ ناتج عن خطأ في القياس فقط
 (ب) الزمن المنقضي يشمل بالإضافة إلى زمن المعالج مكونات أخرى مثل زمن عمليات الإدخال/الإخراج، وأعباء نظام التشغيل، ووقت الخمول أثناء انتظار موارد النظام، بينما زمن المعالج هو الزمن الذي استغرقه المعالج حصراً في معالجة المهمة
 (ج) زمن المعالج دائماً أكبر من الزمن المنقضي لأن المعالج يعالج مهاماً أخرى بالتوازي
 (د) الزمن المنقضي هو نفسه الإنتاجية (Throughput) وليس له أي علاقة بزمن المعالج إطلاقاً

الشرح: الزمن المنقضي يشمل: وقت المعالجة (Processing)، عمليات الإدخال والإخراج، أعباء نظام التشغيل (OS Overhead)، ووقت الخمول (Idle Time)، بينما زمن المعالج (CPU Time) هو الزمن الذي استغرقه المعالج حصراً في معالجة المهمة، وهو أحد مكونات الزمن المنقضي فقط لا كامله.

الأخطاء: افتراض التساوي التام بين المفهومين يتجاهل التفصيل الصريح لأربعة مكونات مختلفة للزمن المنقضي. عكس العلاقة (اعتبار زمن المعالج أكبر من الزمن المنقضي) يخالف منطقياً كون زمن المعالج أحد مكونات الزمن المنقضي وليس أكبر منه. الخلط بين الزمن المنقضي (مدة زمنية) والإنتاجية (معدل إنجاز) يدمج مفهومين مختلفين تماماً في طبيعتهما.

الجواب: ب.

س7: يستبدل مهندس معالج نظام بمعالج أسرع، فيلاحظ أن زمن تنفيذ البرنامج الواحد ينخفض من 10 ثوانٍ إلى 5 ثوانٍ، وأن عدد المهام المنجزة خلال وحدة الزمن يزداد أيضاً. في المقابل، عند إضافة عدة معالجات للنظام بدلاً من معالج واحد أسرع، تزداد الإنتاجية بشكل ملحوظ، لكن زمن استجابة المهمة الفردية الواحدة لا يتحسن بالضرورة. أي استنتاج عام يعكسه هذا السيناريو؟

(أ) تحسين الإنتاجية يؤدي دائماً إلى تحسين زمن الاستجابة والعكس صحيح أيضاً بشكل متماثل تماماً

(ب) لا توجد أي علاقة بين المقياسين، فتحسين أحدهما مستقل تماماً عن الآخر في جميع الحالات

(ج) تحسين زمن الاستجابة يؤدي دائماً إلى تحسين الإنتاجية، لكن العكس ليس صحيحاً دائماً؛ فتحسين الإنتاجية (كإضافة معالجات) لا يضمن بالضرورة تحسين زمن استجابة المهمة الفردية

(د) تحسين زمن الاستجابة يقلل الإنتاجية دائماً لأن المعالج الأسرع يستهلك موارد أكبر من النظام

الشرح: تحسين زمن الاستجابة (Time Response) يؤدي دائماً لتحسين الإنتاجية (Throughput)، ولكن العكس ليس صحيحاً دائماً، وتوضح ذلك بمثال الباص مقابل السيارة الصغيرة مقابل السرفيس السريع، حيث تزيد الإنتاجية بإضافة معالجات دون أن يتحسن بالضرورة زمن استجابة المهمة الفردية.

الأخطاء: افتراض علاقة متماثلة تماماً في الاتجاهين يناقض التحديد الصريح لاتجاه واحد مضموناً فقط. نفي وجود أي علاقة يتجاهل الاتجاه الواحد المضمون (تحسين زمن الاستجابة يحسن الإنتاجية دائماً) الذي مذكور بوضوح. اعتبار العلاقة عكسية (تحسين أحدهما يضر بالآخر) يناقض السيناريو المعطى، الذي يظهر أن المعالج الأسرع حسّن كلا المقياسين معاً.

الجواب: ج.

س8: يشرح أستاذ لطلابه أن الحاسوب لا يعتمد على نوع ذاكرة واحد، بل يستخدم عدة أنواع مرتبة هرمياً بحيث كل مستوى أسرع وأعلى ثمناً من المستوى الذي يليه. ما الترتيب الهرمي الصحيح لأنواع الذاكر من الأعلى (الأسرع والأعلى) إلى الأدنى (الأبطأ والأرخص)؟

أ) RAM ← Cache ← Register ← ROM ← Hard Disk

ب) Hard Disk ← ROM ← RAM ← Cache ← Register

ج) جميع أنواع الذاكر متساوية من حيث السرعة والتكلفة، والاختلاف الوحيد بينها هو السعة التخزينية فقط

د) Register ← Cache ← RAM ← ROM ← Hard Disk

الشرح: الترتيب الهرمي هو: Hard disk - ROM - RAM - cache - register، مع توضيح أن register في قمة الهرم كأفضل خيار لكن لا يمكن استخدامه دائماً نظراً لسعره المرتفع، وال Hard disk وسيط تخزين دائم.

الأخطاء: وضع RAM قبل Cache و Register يعكس الترتيب الصحيح. الافتراض بأن الذاكرة الأكبر سعة يجب أن تكون في القمة يعكس أساس الترتيب أصلاً؛ فالهرمية مبنية على السرعة والتكلفة لا السعة، والسعة في الحقيقة تزداد كلما اتجهنا للأسفل. القول بتساوي جميع الأنواع في السرعة والتكلفة يناقض الفكرة الجوهرية للهرمية بأكملها.

الجواب: د.

س9: فكرتان من ضمن Eight Great Ideas: الأولى تعتمد على توفر أكثر من معالج فعلياً بحيث تُنفَّذ مهمتان مستقلتان في الوقت نفسه دون أن تؤثر إحداها على الأخرى (Multithreading)، والثانية تعتمد على تقسيم تنفيذ المهمة إلى مراحل بحيث يمكن إدخال مهمة جديدة إلى مرحلة تحررت فور انتقال المهمة السابقة إلى المرحلة التالية. أي فكرة تصف كل من الوصفين؟

أ) الوصف الأول يمثل Performance via Parallelism (وهي فكرة متعلقة بال hardware، مثل توفر عدة معالجات لتنفيذ مهام مستقلة بالتوازي)، والثاني يمثل Performance via Pipelining (وهي فكرة متعلقة بال software/التنفيذ، تعتمد على تداخل مراحل تنفيذ المهام)

ب) الوصف الأول يمثل Performance via Pipelining، والثاني يمثل Performance via Parallelism

ج) كلا الوصفين يمثلان الفكرة نفسها (Performance via Parallelism) لأن كليهما يهدف لتسريع تنفيذ عدة مهام

د) لا علاقة بين الوصفين وبين أي من الفكرتين الثمانية؛ فهما يخطان موضوع الذاكرة الهرمية فقط

الشرح: Performance via Parallelism بأنها متعلقة بالhardware (كتوفر معالجين أو أكثر لتنفيذ مهمتين مستقلتين بالتوازي دون أن تؤثر إحداهما على الأخرى)، وPerformance via Pipelining بأنها متعلقة بالsoftware/التنفيذ (تقسيم المهمة لمراحل وإدخال مهمة جديدة إلى مرحلة تحررت فور انتقال المهمة السابقة).

الأخطاء: عكس الفكرتين يناقض التوصيف الحرفي لكل منهما. دمج الوصفين تحت فكرة واحدة يتجاهل التمييز الصريح (hardware مقابل software) بين الفكرتين ضمن قائمتها الثمانية. نفي العلاقة بينهما وبين الأفكار الثمانية يتجاهل أنهما مذكورتان صراحة ضمن هذه القائمة تحديداً.

الجواب: أ.

س10: يلاحظ مبرمج أن المعالج، عند الوصول إلى عبارة شرطية أو حلقة تكرارية، يبدأ أحياناً بتنفيذ التعليمات التالية المتوقعة قبل التأكد الفعلي من نتيجة الشرط. أي فكرة من Eight Great Ideas يصف هذا السلوك، وما الهدف منه؟

أ) Make the Common Case Fast، والهدف تسريع المهام الأكثر استخداماً فقط

ب) Performance via Prediction، والهدف رفع الأداء عبر التنبؤ بالتعليمات القادمة

ج) Abstraction، والهدف فصل المستخدم عن تفاصيل ما يجري داخل الحاسوب

د) Dependability via Redundancy، والهدف توفير نسخ احتياطية من التعليمات المتوقعة

الشرح: Performance via prediction، التنبؤ يعني وضع احتمال لكل شرط (العبارات الشرطية/التكرارية) لمعرفة ما هي التعليمات القادمة التي نريد كتابتها، وتحث هذه الطريقة على استخدام التنبؤ لمعرفة الأحداث التالية مما يرفع الأداء.

الأخطاء: Make the Common Case Fast فكرة مختلفة تماماً تخص تسريع العمليات الأكثر تكراراً، لا التنبؤ بنتائج الشروط. Abstraction تخص فصل المستخدم عن التفاصيل الداخلية، ولا علاقة مباشرة لها بالتنبؤ بالشروط. Dependability via Redundancy تخص الموثوقية عبر النسخ الاحتياطية للحفاظ على البيانات، وهي فكرة منفصلة تماماً عن هدف رفع السرعة عبر التنبؤ.

الجواب: ب.

س11: مستخدم يشغل تطبيقاً على هاتفه دون أي معرفة بكيفية تحويل نقرات الشاشة إلى إشارات كهربائية داخل الشرائح الإلكترونية. أي فكرة من Eight Great Ideas تفسر هذا الفصل بين تجربة المستخدم والتفاصيل الداخلية للعتاد، وما شعارها المختصر؟

- (أ) Make the Common Case Fast، وشعارها 'اجعل المهام المتكررة أسرع'
 (ب) Performance via Pipelining، وشعارها 'قسّم المهمة إلى مراحل متداخلة'
 (ج) Abstraction، وشعارها 'استخدم التجريد لتبسيط التصميم' (Use Abstraction To Simplify Design)
 (د) Hierarchy of Memories، وشعارها 'استخدم عدة أنواع من الذاكر معاً'

الشرح: التجريد Abstraction يعمل على فصل المستخدم user عن ما يجري داخل الحاسوب، التجريد يعني التبسيط: Use Abstraction To Simplify Design.

الأخطاء: كل من الأفكار الثلاث الأخرى (تسريع المهام المتكررة، تداخل مراحل التنفيذ، هرمية الذاكر) موصوفة بتعريفها الخاص المستقل، ولا يطابق أي منها الوصف المحدد في السؤال (إخفاء التعقيد الداخلي عن المستخدم)، والذي هو التعريف الحصري لفكرة التجريد تحديداً.

الجواب: ج.

س12: يعرض أستاذ مخططاً مفصلاً لطبقات النظام الحاسوبي يتضمن الترتيب التالي من الأعلى إلى الأسفل: Application (خوارزميات ولغة برمجة) ← Operating System/Virtual ← Compiler/Linker/Runtime/Libraries ← Logic Gates/Circuits ← Micro Architecture ← Instruction Set Architecture (ISA) ← Machine ← Physics Devices/Layout. بالمقارنة مع الوصف المبسط لموقع ISA (كحد فاصل بين طبقة برمجيات عليا وطبقة عتاد سفلى)، أي وصف يعكس بدقة موقع ISA ضمن هذا المخطط الموسع الوارد؟

(أ) ISA تحل محل طبقتي المترجم (Compiler) ونظام التشغيل بالكامل، فلم تعد هاتان الطبقتان ضروريتين في هذا المخطط

(ب) ISA تقع في أعلى المخطط فوق طبقة Application مباشرة

(ج) ISA غير موجودة إطلاقاً ضمن هذا المخطط الموسّع؛ فقد تم استبدالها بالكامل بطبقة Logic Gates and Circuits

(د) ISA تقع أسفل طبقة نظام التشغيل (Operating System/Virtual Machine) مباشرة وفوق طبقة Micro Architecture، فهي الحد الفاصل الدقيق بين ما يفهمه الحاسوب من تعليمات وبين تفاصيل تنفيذها الدقيقة داخل العتاد

الشرح: يسرد المخطط الموسَّع الترتيب التالي: ← Application

← OS/Virtual Machine ← ISA ← Micro

← Physics ← Devices/Layout ← Logic Gates/Circuits ← Architecture، أي أن ISA تقع تحديداً بين طبقة نظام التشغيل من فوق وطبقة الـ Micro Architecture من تحت.

الأخطاء: لا يشير المخطط إلى إلغاء طبقتي Compiler أو نظام التشغيل؛ فكلتاها مذكورتان صراحة كطبقات منفصلة أعلى ISA. وضع ISA أعلى طبقة Application يعكس الترتيب الفعلي الوارد في المخطط تماماً. إنكار وجود ISA ضمن المخطط يناقض إدراجها الصريح بين الطبقات المذكورة.

الجواب: د.

س13: عند فتح الغطاء الخارجي لجهاز iPad 2 من Apple وتشريح معالج A5 بداخله، يحدد المهندسون ثلاثة مكونات رئيسية داخل وحدة المعالجة المركزية. ما هذه المكونات الثلاثة ووظيفة كل منها؟

أ) Input Unit، Output Unit، Storage Unit، وهي المكونات الثلاثة الأساسية لأي معالج بغض النظر عن الشركة المصنعة

ب) Datapath (وحدة الحساب والمنطق ALU لتنفيذ العمليات على البيانات)، Control (يتحكم بالذاكرة وتنفيذ العمليات)، Cache/Registers (ذاكر سريعة وصغيرة من نوع SRAM مسؤولة عن الوصول الفوري للبيانات)

ج) Compiler، Assembler، Linker، وهي البرمجيات المسؤولة عن ترجمة الكود قبل وصوله إلى المعالج

د) RAM، ROM، Hard Disk، وهي أنواع الذاكر الموجودة داخل وحدة المعالجة المركزية حصراً

الشرح: ماذا يوجد ضمن CPU؟ وحدة الحساب والمنطق + وحدة التحكم + registers. Data path هي وحدة الحساب والمنطق ALU تُنجز العمليات على البيانات. Control تتحكم بالذاكرة وتنفيذ العمليات. Cache memory و registers هي ذاكر سريعة وصغيرة الحجم من نوع SRAM مسؤولة عن الوصول الفوري للبيانات.

الأخطاء: Input/Output/Storage Units مكونات عامة لأي نظام حاسوبي، لكنها ليست التوصيف المحدد لما بداخل CPU تحديداً بحسب هذا التشريح. Compiler و Assembler و Linker برمجيات ترجمة تعمل قبل وصول الكود للمعالج، وليست مكونات عتادية داخل CPU. RAM و ROM والقرص الصلب ذاكر خارجية أو منفصلة عن CPU، بينما المحدد هو Cache/Registers فقط كذاكرة داخلية للمعالج.

الجواب: ب.

س14: يكتب مبرمج برنامجاً بلغة C، ويريد فهم المراحل التي يمر بها هذا الكود حتى يصبح قابلاً للتنفيذ الفعلي من قبل دارات المعالج. ما التسلسل الصحيح لمراحل الترجمة؟

(أ) الـ Assembler يترجم مباشرة من لغة C إلى machine code دون الحاجة لأي مرحلة وسيطة من نوع Assembly

(ب) الـ Compiler يترجم اللغة العالية المستوى إلى لغة Assembly (بعد الربط وكشف الأخطاء وإحضار المكتبات المستخدمة)، ثم يقوم الـ Assembler بتحويل لغة Assembly إلى أرقام ووحدات (machine code) تُفَعَّل دارات التنفيذ مباشرة

(ج) الـ Compiler يحوّل الكود مباشرة إلى إشارات كهربائية فعلية دون المرور بأي تمثيل رقمي وسيط

(د) يبدأ التسلسل بلغة machine code أولاً، ثم يقوم الـ Compiler بترجمتها إلى لغة C ليفهمها المبرمج

الشرح: الـ compiler يقوم بترجمة اللغة عالية المستوى إلى لغة Assembly بعد ربط وكشف الأخطاء إضافة إلى إحضار نسخة من المكتبات المستخدمة في البرنامج، ثم يقوم الـ Assembler بتحويل لغة Assembly إلى أرقام ووحدات (machine code) تُفَعَّل دارات التنفيذ.

الأخطاء: تخطي مرحلة الـ Assembly الوسيطة يناقض التسلسل الصريح ذا المراحل الثلاث الواضحة. افتراض تحويل مباشر إلى إشارات كهربائية يتجاهل مرحلة التمثيل الرقمي (machine code) الوسيطة الضرورية. عكس اتجاه الترجمة بالكامل يناقض منطق التسلسل من اللغة عالية المستوى نزولاً إلى لغة الآلة كما يصفه العنوان نفسه (Top-down).

الجواب: ب.

س15: يسأل طالب عن التاريخ التقريبي الذي بدأ فيه معدل تناؤل حجم الترانزستورات بالتباطؤ عن وتيرته الكلاسيكية (كل سنة ونصف إلى سنتين). ما الإجابة الدقيقة؟

(أ) توقف قانون مور نهائياً منذ عام 1990 ولم يعد له أي أثر بعد ذلك

(ب) لم يُذكر أي تاريخ محدد لبداية هذا التباطؤ

(ج) منذ عام 2003 أصبحت نسبة تناؤل عدد الترانزستورات خلال كل سنة ونصف أقل من ذي قبل، لكن الحجم ما زال يتضاءل

(د) بدأ التباطؤ فور صدور أول معالج تجاري عام 1971 مباشرة

الشرح: حسب الدراسات العلمية ما زال حجم الترانزستورات يتضاعف، لكن منذ عام 2003 أصبحت نسبة تضاعف عدد الترانزستورات خلال كل عام ونصف أقل.

الأخطاء: الادعاء بتوقف كامل ونهائي عام 1990 يناقض الإطار الذي يصف تباطؤاً جزئياً لا توقفاً كاملاً، ويستخدم تاريخاً مختلفاً تماماً عن التاريخ المذكور فعلياً (2003). إنكار وجود تاريخ محدد يتجاهل الذكر الصريح لعام 2003. الربط بعام 1971 (صدر أول معالج صغري) يخلط بين حدثين تاريخيين منفصلين تماماً غير مرتبطين بهذا السياق.

الجواب: ج.

س16: يشرح أستاذ سبب حرص مصممي المعالجات على تحديد استهلاك الترانزستورات للكهرباء ضمن حدود معينة، حيث يحذر من أن تتجاوز هذه الحدود يؤدي لمشكلة فيزيائية حرجية. ما هذه المشكلة الفيزيائية الحرجية بحسب شرح قانون دينارد؟

أ) استهلاك الكهرباء الزائد يؤدي مباشرة إلى تلف دارات الذاكرة العشوائية RAM فقط دون أي تأثير على المعالج نفسه

ب) لا علاقة بين استهلاك الكهرباء ودرجة حرارة الشريحة؛ فالمشكلة الوحيدة هي ارتفاع تكلفة الإنتاج

ج) زيادة استهلاك الكهرباء تزيد سرعة المعالج دائماً دون أي أثر جانبي سلبي يُذكر

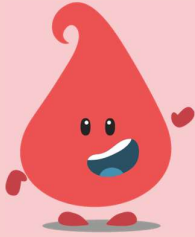
د) كلما استهلك الترانزستور كهرباء أكثر، ارتفعت درجة حرارة الشريحة حتى تصل لدرجة لا يستطيع فيها مبرد المعالج خفض حرارتها

الشرح: قانون دينارد يركز على توفير استهلاك الطاقة، وينص على أن الترانزستورات يجب أن يكون استخدامها للكهرباء محدوداً، حيث كلما استهلك الترانزستور كهرباء أكثر كلما ارتفعت درجة حرارة الشريحة لدرجة أن مبرد المعالج لن يقدر على خفض درجة حرارتها.

الأخطاء: حصر التأثير في دارات RAM فقط يخالف الوصف العام للمحاضرة الذي يتحدث عن الشريحة ككل. نفي أي علاقة بين استهلاك الكهرباء والحرارة يناقض جوهر قانون دينارد بالكامل. الادعاء بأن زيادة استهلاك الكهرباء بلا أي أثر سلبي يناقض سبب وجود هذا القانون أصلاً، وهو تجنب مشاكل الحرارة الناتجة عن ذلك بالضبط.

الجواب: د.

س17: نظام تحكم صغير مدمج داخل سيارة ذكية، مبرمج خصيصاً لأداء مهمة محددة (كالتحكم بالفرامل)، ولا يظهر للمستخدم كجهاز حاسوب مستقل بحد ذاته. أي فئة من فئات الحواسيب يصفها هذا المثال، وما أبرز خصائصها؟



(أ) الحواسيب المدمجة (Embedded Computers)

(ب) المخدمات (Servers)

(ج) الحواسيب الشخصية (Personal Computers)

(د) الحواسيب الفائقة (Supercomputers)

الشرح: الحواسيب المدمجة (Embedded Computers) بأنها معالجات صغيرة تُضمّن ضمن أنظمة أخرى، تتميز بأنها تختفي كجزء من نظام معين، أداؤها وتكلفتها بسيطتان، مبرمجة خصيصاً لأداء مهمة محددة، مثل السيارات الذكية، مع استهلاك طاقة قليل نظراً لحجمها الصغير ووظيفتها المحدودة.

الأخطاء: المخدمات (Servers) فئة تخص تقديم خدمات موثوقة واسعة النطاق، لا التحكم بمهمة صغيرة مخفية داخل نظام آخر. الحواسيب الشخصية فئة أجهزة متعددة الاستخدامات موجهة مباشرة للمستخدم النهائي، بعكس النظام المخفي الموصوف في السيناريو. الحواسيب الفائقة مخصصة لقدرات حسابية علمية ضخمة، وهي خاصة مختلفة تماماً عن صغر الحجم والوظيفة المحدودة التي تتميز الحواسيب المدمجة.

الجواب: أ.

س18: يقارن طالب بين حاسوب شخصي تقليدي وجهاز محمول حديث مثل الساعة الذكية أو النظارة الإلكترونية. أي مجموعة من الخصائص التالية تصف بدقة فئة PMD (Personal Mobile Devices)؟

(أ) تعمل حصراً بتوصيلها بمصدر كهرباء ثابت، ولا يمكنها الاتصال بأي شبكة إنترنت إطلاقاً

(ب) تعمل على البطارية، قادرة على الاتصال بشبكة الإنترنت، يصل سعرها إلى مئات الدولارات، وتشمل أنواعاً متعددة مثل الهواتف الذكية والساعات الذكية والنظارات الإلكترونية والأجهزة اللوحية

(ج) أسعارها تتجاوز دائماً آلاف الدولارات لأنها أكثر تطوراً من الحواسيب الشخصية التقليدية

(د) هي فئة مطابقة تماماً لفئة الحواسيب الفائقة (Supercomputers) من حيث القدرة الحسابية والاستخدام

الشرح: بدأت بالأجهزة المحمولة PMD: تعمل على البطارية، قادرة على الاتصال بشبكة الإنترنت، يصل سعرها إلى مئات الدولارات، ويوجد منها أنواع عديدة مثل smart watches، electronic tablets, glasses.

الأخطاء: نفى إمكانية الاتصال بالإنترنت والاعتماد الحصري على مصدر كهرباء ثابت يناقض الوصف الصريح (تعمل على البطارية وقادرة على الاتصال بالإنترنت). المبالغة في السعر (آلاف الدولارات) تخالف النطاق المذكور فعلياً (مئات الدولارات). مساواة PMD بالحواسيب الفائقة تخلق بين فئتين مختلفتين تماماً من حيث الحجم والقدرة الحسابية والاستخدام.

الجواب: ب.

س19: يعرض أستاذ مثلاً لأربع شركات طيران، كل منها تتفوق في ميزة مختلفة (السعر، السرعة، الراحة، عدد الوجهات)، ويسأل الطلاب أي شركة يجب أن يختاروها. ما الفكرة الأساسية التي يوضحها هذا المثال فيما يخص تقييم أداء الحاسوب؟

- (أ) يوجد دائماً مقياس واحد موضوعي وثابت يحدد الأداء الأفضل في جميع الحالات دون استثناء
(ب) لا علاقة بين مثال شركات الطيران وموضوع أداء الحاسوب إطلاقاً؛ فهو مجرد مثال ترفيهي
(ج) لا يمكن الحكم على الأداء من وجهة نظر واحدة فقط؛ فالمقياس الأنسب لتقييم الأداء يعتمد على حاجة المستخدم ومتطلباته تحديداً
(د) أفضل شركة طيران هي الأرخص دائماً بغض النظر عن أي معيار آخر، وينطبق الأمر نفسه على أداء الحاسوب (الأرخص هو الأفضل دائماً)

الشرح: خلاصة هذا المثال: لا يمكنني الحكم على الأداء من وجهة نظر واحدة، ويحدد الأداء الأنسب حسب الحاجة، وذلك تمهيداً لمناقشة معايير أداء الحاسوب المتعددة.

الأخطاء: افتراض وجود مقياس واحد موضوعي ثابت يناقض الاستنتاج الصريح للمحاضرة من هذا المثال بالذات. اعتبار المثال غير ذي صلة يتجاهل غرضه التمهيدي الواضح في افتتاح موضوع الأداء. اختزال المعيار في السعر فقط يمثل بالضبط نوع الحكم أحادي الجانب الذي يحذر منه المثال.

الجواب: ج.

س20: يقترح طالب استخدام مقياس MIPS فقط للمقارنة بين أي حاسوبين، بغض النظر عن بنيتهما (Architecture) أو البرنامج المُشغَّل عليهما. ما أبرز الانتقادات الموجهة لهذا الاقتراح بحسب عيوب MIPS؟

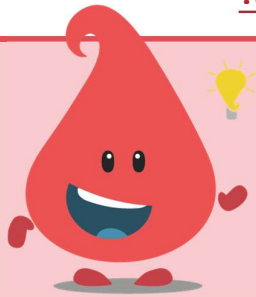
- (أ) لا توجد أي عيوب فعلية لمقياس MIPS، وهو مقياس دقيق وكافٍ تماماً بمفرده في كل الحالات
- (ب) عيب واحد فقط: أن المقياس معقد الحساب رياضياً بشكل يصعب على أغلب المهندسين استخدامه
- (ج) عيب واحد فقط: أن وحدة القياس (مليون تعليمة بالثانية) كبيرة جداً بحيث لا يمكن تطبيقها على المعالجات الحديثة
- (د) MIPS لا يراعي قدرة/تعقيد التعليمة الواحدة عند مقارنة معماريات مختلفة، وقيمه تتغير بتغير البرنامج المُشغَّل (فلا يوجد رقم MIPS ثابت للجهاز الواحد)، وقد يتغير بعلاقة عكسية مع الأداء الفعلي كما يحدث أحياناً

الشرح: عيوب MIPS الثلاثة هي: عدم مراعاة قدرة/تعقيد التعليمة عند مقارنة معماريات مختلفة، تغيير قيمة MIPS بتغير البرنامج (لا يوجد رقم ثابت للجهاز الواحد)، والعلاقة العكسية المحتملة مع الأداء الفعلي.

الأخطاء: إنكار وجود أي عيوب يناقض تخصيص قسم كامل بعنوان صريح 'Drawbacks of MIPS'. اختزال العيوب في تعقيد الحساب الرياضي يخترع انتقاداً غير مذكور إطلاقاً. اختزال العيوب في حجم وحدة القياس نفسها يخترع انتقاداً آخر غير وارد أيضاً في قائمة العيوب الفعلية الثلاثة.

الجواب: د.

س21: يملك معالج معين دورة ساعة (Clock Period) مقدارها 250 بيكو ثانية (0.25 نانوثانية). ما تردد الساعة (Clock Rate) المقابل لهذا المعالج بحسب العلاقة $(\text{clock rate} = 1/\text{clock period})$ ؟



(أ) 4 جيجا هرتز (4GHz)

(ب) 0.25 جيجا هرتز

(ج) 250 ميغا هرتز

(د) 25 جيجا هرتز

الشرح: بتطبيق القانون مباشرة: $\text{clock rate} = 1/\text{clock period} = 1/(250 \times 10^{-12} \text{ s}) = 4 \times 10^9 = 4 \text{ جيجا هرتز}$

الأخطاء: معاملة رقم الدورة (250 أو 0.25) على أنه هو نفسه قيمة التردد يتجاهل أن العلاقة بينهما علاقة مقلوب (reciprocal) لا تطابق مباشر، كما هو مذكور صراحة. الخلط بين البيكوثانية (10^{-12}) والميكروثانية (10^{-6}) عند التحويل يُنتج قيمة أصغر بمقدار كبير من القيمة الصحيحة. خطأ موضع الفاصلة العشرية عند القسمة يُنتج قيمة أكبر بعشر مرات من الناتج الصحيح.

الجواب: أ.

س22: شركة صغيرة تستأجر مساحة تخزين ومعالجة من شركة تقنية كبرى بدلاً من شراء عتاد خاص بها، وتستخدم أيضاً برنامج محاسبة يُدفع اشتراكه شهرياً عبر الإنترنت دون الحاجة لتثبيته محلياً. أي المفاهيم يصف كل جزء من هذا السيناريو؟

(أ) كلا جزأي السيناريو يمثلان WSC حصراً، ولا علاقة لأي منهما بمفهوم SaaS

(ب) استئجار التخزين يمثل SaaS، بينما برنامج المحاسبة يمثل WSC

(ج) لا ينطبق أي من هذين المفهومين (Cloud أو SaaS) على هذا السيناريو؛ فكلاهما يخص فقط الشركات الكبرى وليس الشركات الصغيرة المستأجرة

(د) استئجار مساحة التخزين والمعالجة يمثل الحوسبة السحابية (Cloud/WSC)، بينما برنامج المحاسبة المدفوع اشتراكه يمثل SaaS

الشرح: WSC بأنها وسائط تخزين ضخمة مرتبطة بحواسيب ذات إمكانيات عالية (سيرفرات) يمكن استئجارها بدلاً من شرائها لتقديم خدمات معالجة وتخزين للأفراد والشركات، وتصف SaaS بأنها برمجيات يتم استخدامها أو شراؤها ولها حقوق نشر وصلاحيات معينة، تعمل جزئياً على السحابة (cloud) التي تُستأجر أو تُشتري.

الأخطاء: دمج كلا جزأي السيناريو تحت WSC وحده يتجاهل أن SaaS تُعرّف كمفهوم مستقل تماماً عن البنية التحتية (WSC)، يخص البرمجيات المستخدمة كخدمة مدفوعة الاشتراك تحديداً. عكس التصنيف بين المفهومين يبدّل بين مفهوم البنية التحتية المستأجرة (WSC) ومفهوم البرمجية المستأجرة (SaaS). استبعاد الشركات الصغيرة من الاستفادة من هذين المفهومين يناقض الفكرة الأساسية وراء انتشارهما أصلاً، وهي إتاحة موارد ضخمة للشركات الصغيرة دون الحاجة لامتلاكها.

الجواب: د.

المحاضرة 3: لغة الحاسوب

س1: مهندس يبدأ بتصميم مجموعة تعليمات (ISA) جديدة لمعالج، ويسأل زميله عن العناصر الإلزامية التي يجب تحديدها قبل اعتماد التصميم النهائي. ما العناصر الأربعة الأساسية التي يجب تحديدها عند تصميم أي ISA؟

أ) السجلات (Registers) وتصنيفها وتسميتها، أنماط نقل البيانات بين السجلات والذاكرة ووحدات الإدخال/الإخراج، كفاءة التنفيذ (احتواء تعليمات كافية لترجمة أي برنامج بكفاءة)، وتنسيق التعليمات (التمثيل الثنائي وكونها ثابتة أم متغيرة الطول)

ب) نوع الذاكرة المستخدمة فقط (RAM أو Cache)، بغض النظر عن أي عناصر أخرى

ج) سعر المعالج، واستهلاكه للطاقة، وحجمه الفيزيائي فقط دون أي اعتبارات تقنية أخرى

د) عدد الأنوية (Cores) المتوفرة في المعالج فقط

الشرح: هناك أربعة عناصر يجب تحديدها عند تصميم أي ISA: السجلات وتصنيفها وتسميتها، أنماط نقل البيانات بين السجلات والذاكرة ووحدات الإدخال/الإخراج، كفاءة التنفيذ (احتواء تعليمات كافية لترجمة أي برنامج بكفاءة إلى لغة الآلة)، وتنسيق التعليمات (التمثيل الثنائي وثبات أو تغير الطول).

الأخطاء: كل من الخيارات الثلاثة الأخرى يختزل التصميم في عامل واحد ضيق غير مرتبط (أو مرتبط جزئياً فقط) بالعناصر الأربعة الشاملة المحددة صراحة، متجاهلاً القائمة الكاملة (سجلات، أنماط نقل بيانات، كفاءة، تنسيق تعليمات).

الجواب: أ.

س2: معالج يحتوي على مئات التعليمات المختلفة، حيث قد تستغرق التعليمية الواحدة عدة دورات ساعة لتنفيذها بالكامل، ويُستخدم هذا النوع في معظم أجهزة الحاسوب المكتبي والمحمول. أي معمارية يصفها هذا المثال، وما اسم اختصارها الكامل؟

أ) RISC، اختصاراً لـ Reduced Instruction Set Computer، وتتميز بمجموعة تعليمات صغيرة تنفذ في دورة واحدة تقريباً

ب) CISC، اختصاراً لـ Complex Instruction Set Computer، وتتميز بامتلاكها عدة مئات من التعليمات المعقدة التي قد يستغرق تنفيذ الواحدة منها عدة دورات، مثل معالجات Intel x86

ج) ARM، وهي معمارية مخصصة حصراً للهواتف الذكية ولا علاقة لها بعدد التعليمات أو تعقيدها

د) MIPS، وهي معمارية تعتمد بشكل حصري على تعليمات واحدة فقط للقيام بكل المهام

الشرح: حاسوب مجموع التعليمات المعقدة (CISC (Complex Instruction Set Computer، لديه العديد من التعليمات (عدة مئات)، والتعليمات فيه معقدة، أي الأمر الواحد يمكن أن يستغرق تنفيذه عدة دورات، مثال: معالجات Intel x86 الموجودة في معظم أجهزة اللابتوب والكمبيوتر المكتبي.

الأخطاء: الخيار الأول يصف خصائص RISC المعاكسة تماماً لما ورد في السيناريو. الخيار الثالث يسيء تصنيف ARM بوصفه غير مرتبط بعدد التعليمات أو تعقيدها، بينما يُذكر ARM لاحقاً كمثال على RISC تحديداً. الخيار الرابع يخترع خاصية غير واقعية (تعليمية واحدة فقط) لا تصف أي معمارية حقيقية ذُكرت.

الجواب: ب.

س3: معالج يحتوي على مجموعة صغيرة نسبياً من التعليمات (حوالي 32 تعليمة)، كل تعليمة مصممة لأداء مهمة واحدة بسيطة، وتنفذ التعليمة الواحدة في دورة ساعة واحدة تقريباً، مع استخدام فعال لتقنية التوارد (Pipelining). أي معمارية يصفها هذا المثال، وما مثالان عليها وردا؟

أ) CISC، ومثالها Intel x86 و IBM 360

ب) معمارية المكسد (Stack Architecture)، ومثالها ARM و MIPS

ج) RISC، اختصاراً لـ Reduced Instruction Set Computer، ومثالها ARM (المستخدم في الهواتف الذكية) و MIPS

د) معمارية المراكم (Accumulator Architecture)، ومثالها ARM و Intel x86

الشرح: حاسوب مجموع التعليمات المختزلة (RISC (Reduced Instruction Set Computer، مجموعة بسيطة وصغيرة من التعليمات (عادة 32 تعليمة)، كل أمر مصمم ليقوم بمهمة واحدة بسيطة، كل تعليمة تنفذ في دورة ساعة واحدة تقريباً، مع استخدام فعال لتقنية التوارد؛ مثال: ARM الموجود في الهواتف الذكية، و MIPS.

الأخطاء: الخيار الأول يسيء تصنيف الوصف على أنه CISC، وهي معمارية بخصائص معاكسة تماماً (مئات التعليمات المعقدة متعددة الدورات). الخيار الثاني والرابع يخلطان بين RISC (تصنيف حسب تعقيد وعدد التعليمات) وبين Stack/Accumulator Architecture (تصنيف حسب مكان المعاملات، مفهوم منفصل تماماً يُشرح لاحقاً).

الجواب: ج.

س4: لدينا 10 صناديق نريد نقلها من طرف الطريق إلى الطرف الآخر. الخيار الأول: شاحنة ضخمة تحمل الصناديق العشرة معاً لكنها بطيئة الحركة. الخيار الثاني: دراجة تحمل صندوقاً واحداً فقط في كل رحلة لكنها سريعة جداً، فتذهب وتعود عدة مرات. أي معمارية يمثلها كل خيار من الخيارين بحسب التشبيه؟

(أ) الشاحنة تمثل RISC، والدراجة تمثل CISC

(ب) كلاهما يمثلان RISC

(ج) لا علاقة لهذا التشبيه بمعماريتي CISC و RISC؛ فهو مجرد مثال توضيحي منفصل

(د) الشاحنة تمثل CISC (تنفيذ معقد وبطيء يحمل كمّاً كبيراً من العمل دفعة واحدة)، والدراجة تمثل RISC (تنفيذ بسيط وسريع يتكرر عدة مرات لإنجاز المهمة نفسها)

الشرح: يوضح هذا المثال ذلك: الشاحنة الضخمة القادرة على حمل الصناديق العشرة معاً لكنها بطيئة الحركة تمثل CISC، بينما الدراجة القادرة على حمل صندوق واحد فقط لكنها سريعة جداً وستذهب وتعود أكثر من مرة تمثل RISC.

الأخطاء: الخيار الأول يعكس المطابقة الصحيحة بين وسيلتي النقل ومعماريتهما. الخيار الثاني يتجاهل الاختلاف الجوهرى بين الوسيلتين الذي يوضحه المثال أصلاً (سعة النقل مقابل السرعة). الخيار الثالث ينكر الغرض التوضيحي الصريح لهذا المثال ضمن سياق شرح فائدة RISC.

الجواب: د.

س5: يقرأ طالب في مصادر مختلفة مصطلحات مثل Wall-clock time و Elapsed time و Response time، ويتساءل إن كانت هذه المصطلحات تشير إلى مفاهيم مختلفة تماماً أم لا. ما العلاقة بين هذه المصطلحات؟

(أ) هي جميعها مسميات مختلفة تشير إلى المفهوم نفسه، وهو زمن التنفيذ (Execution Time)

(ب) كل مصطلح يشير إلى مفهوم منفصل تماماً عن الآخرين، ولا علاقة بينها إطلاقاً

(ج) Wall-clock time و Elapsed time مترادفان، لكن Response time يشير حصراً إلى زمن استجابة الشبكة فقط

(د) هذه المصطلحات غير مذكورة إطلاقاً

الشرح: ويوجد مسميات أخرى لزمان التنفيذ: Response time, Elapsed time, Wall-clock time.

الأخطاء: الخيار الثاني والثالث يخترعان فروقاً غير موجودة بين مصطلحات تُعامل كمتبادلات تماماً.
الخيار الرابع ينكر الذكر الصريح لهذه المصطلحات الثلاثة تحديداً في هذا السياق بالذات.

الجواب: أ.

س6: معالج يعمل بتردد 3 جيجا هرتز، أي أنه ينجز 3 مليارات دورة في الثانية الواحدة. أي عاملين يحددان معاً سرعة تنفيذ برنامج معين على هذا المعالج؟

- (أ) عدد التطبيقات المفتوحة على الجهاز فقط، بغض النظر عن التردد أو عدد الدورات
- (ب) عدد الدورات (cycles) اللازمة لتنفيذ البرنامج والتردد (clock rate)؛ فكلما قل عدد الدورات وزاد التردد كان التنفيذ أسرع
- (ج) حجم الذاكرة الخبيئة (Cache) فقط، بغض النظر عن أي عامل آخر متعلق بالساعة
- (د) نوع نظام التشغيل المُستخدم حصراً، دون أي اعتبار لتردد المعالج أو عدد الدورات

الشرح: سرعة تنفيذ البرنامج تعتمد على عدد cycles اللازمة للتنفيذ، وبالتالي كلما قل عدد cycles وزاد التردد كان التنفيذ أسرع.

الأخطاء: الخيارات الثلاثة الأخرى تستبدل العاملين المحددين صراحة (عدد الدورات والتردد) بعوامل أخرى غير مذكورة في هذا السياق (عدد التطبيقات، حجم الكاش، نوع نظام التشغيل).

الجواب: ب.

في الحقيقة أنت من تصنع الفرص ...

و ليس العكس !!

س7: ثلاثة معالجات مختلفة تنفذ العملية نفسها ($\alpha + b = c$) بثلاث طرق مختلفة: المعالج الأول يجمع القيمة الموجودة في قمة المكدر (TOS) مع القيمة التالية ويضع الناتج في القمة. المعالج الثاني يحقّل القيمة A في سجل خاص (المراكم)، يضيف إليه B، ويخزّن الناتج في السجل نفسه. المعالج الثالث يضع A في سجل R1 و B في سجل R2، ثم يجمعهما ويضع الناتج في سجل R3. ما اسم كل نمط من الأنماط الثلاثة؟

أ) المعالج الأول: General Purpose Register، الثاني: Stack Architecture، الثالث: Accumulator Architecture

ب) المعالج الأول: Accumulator Architecture، الثاني: General Purpose Register، الثالث: Stack Architecture

ج) المعالج الأول: Stack Architecture، الثاني: Accumulator Architecture، الثالث: General Purpose Register

د) الأنماط الثلاثة تصف المعمارية نفسها (RISC) بثلاث طرق مختلفة للشرح فقط

الشرح: تُوصف الأنماط الثلاثة بدقة كالتالي: Stack Architecture تجمع القيمة في قمة المكدر (TOS) مع القيمة التالية وتضع الناتج في TOS، وتعتمد على دفع وسحب البيانات من قمة المكدر. Accumulator Architecture تعتمد على سجل واحد رئيسي (المراكم) يحمل أحد طرفي أي عملية. General Purpose Register هو النظام المستخدم في المعالجات الحديثة، حيث يضع المعالج A و B في سجلين مرقّمين ويجمعهما في سجل ثالث.

الأخطاء: الخيار الأول والثاني يبدّلان ترتيب المطابقة الصحيحة بين سلوك كل معالج واسم المعمارية المناظرة له. الخيار الرابع يخلط بين RISC (تصنيف حسب تعقيد التعليمات) وبين الأنماط الثلاثة لتنظيم العمل (تصنيف حسب مكان المعاملات)، وهما محوران تصنيفيان مستقلان تماماً.

الجواب: ج.

س8: يقارن طالب بين الأنماط الثلاثة لتنظيم العمل من حيث مكان تخزين المعاملات أثناء تنفيذ العملية. أين تكون المعاملات (Operands) موجودة حصراً في معمارية المكدر (Stack Architecture)؟

أ) في السجلات (Registers) فقط، تماماً كما في معمارية GPR

ب) في السجلات والذاكرة معاً، تماماً كما في معمارية المراكم

ج) في وحدات الإدخال والإخراج مباشرة، دون المرور بالذاكرة أو السجلات إطلاقاً

د) في الذاكرة فقط

الشرح: تختلف هذه الأنماط في مكان وجود المعاملات، ففي معمارية المكسدس (Stack) تكون في الذاكرة فقط، بينما في معمارية سجلات الأغراض العامة (GPR) تكون في السجلات، وفي معمارية المراكم (Accumulator) قد تكون في كليهما (الذاكرة والسجلات).

الأخطاء: الخيار الأول والثاني ينسبان لمعمارية المكسدس موقع معاملات يخص معماريتي GPR أو المراكم بدلاً من موقعها الخاص (الذاكرة فقط). الخيار الثالث يخترع موقعاً (وحدات الإدخال/الإخراج) غير مذكور إطلاقاً ضمن نقاش موقع المعاملات.

الجواب: د.

س9: يكتب مبرمج العبارة الرياضية $\alpha + b = c$ بلغة C، ثم يريد فهم الشكل الذي ستظهر عليه هذه العملية بعد ترجمتها إلى لغة Assembly بواسطة المترجم (Compiler). أي صيغة Assembly تعكس بدقة ترتيب المعاملات كما في المثال $(add \$s1, \$s2, \$s3)$ ؟

(أ) الترتيب هو: الوجهة أولاً ثم المصدر الأول ثم المصدر الثاني، أي أن $s1$ هي وجهة الناتج، و $s2$ و $s3$ هما مصدرا الجمع

(ب) الترتيب هو: المصدران أولاً ثم الوجهة أخيراً، أي أن $s1$ و $s2$ مصدران، و $s3$ هي وجهة الناتج

(ج) لا يوجد ترتيب ثابت لمعاملات تعليمات Assembly؛ فالترتيب يتغير عشوائياً بين التعليمات

(د) الوجهة والمصدر الأول متطابقان دائماً في نفس السجل، والمصدر الثاني فقط هو المختلف

الشرح: يوضح صراحة بجانب مثال $add \$s1, \$s2, \$s3$ الترتيب التالي: الوجهة $s1$ ، مصدر $s2$ و $s3$.

الأخطاء: الخيار الثاني يعكس الترتيب الصحيح تماماً (يضع الوجهة أخيراً بدلاً من أولاً). الخيار الثالث ينكر الترتيب الثابت الصريح الموضح بجانب المثال مباشرة. الخيار الرابع يخترع قيداً غير موجود إطلاقاً في الصيغة العامة الموضحة.

الجواب: أ.

س10: يسأل طالب عن معنى الاختصار MIPS وعن الجهة التي طورت أول معالج بهذا الاسم. ما الإجابة الدقيقة؟

(أ) MIPS اختصار لـ Million Instructions Per Second فقط، وطُوّر أول معالج بهذا الاسم في شركة Intel عام 1990

(ب) MIPS اختصار لـ Microprocessor without Interlocked Pipelined Stages، وطُوّر أول معالج بهذا الاسم عام 1981 على يد جون هينيسي (John Hennessy) وطلابه في جامعة ستانفورد، وتملكه حالياً شركة Imagination Technologies

(ج) MIPS اختصار لـ Modern Instruction Processing System، وطُوّر عام 2005 في جامعة MIT

(د) لم يرد أي تفصيل حول أصل الاسم أو تاريخ التطوير

الشرح: MIPS اختصار لـ Microprocessor without Interlocked Pipelined Stages، وقد طُوّر أول معالج MIPS عام 1981 من قبل جون هينيسي وطلابه في جامعة ستانفورد، وحالياً تملكه شركة Imagination Technologies.

الأخطاء: الخيار الأول يخلط بين اسم المعمارية MIPS ومقياس الأداء غير المرتبط (Million Instructions Per Second) الذي وردت تفاصيله في محاضرة سابقة، ويخترع شركة وتاريخاً خاطئين. الخيار الثالث يخترع اسماً وأصلاً تاريخياً خياليين تماماً. الخيار الرابع ينكر التفاصيل التاريخية الصريحة المذكورة بوضوح.

الجواب: ب.

س11: تُستخدم معالجات MIPS على نطاق واسع في أجهزة يومية مثل الطابعات والكاميرات ومعدات الشبكات.

ما الأهمية الإضافية لدراسة MIPS تحديداً، بالإضافة إلى انتشاره في هذه الأجهزة؟

(أ) لا توجد أي أهمية إضافية؛ فهو معالج نادر الاستخدام مقارنة بغيره

(ب) أهميته تقتصر فقط على كونه أرخص المعالجات المتوفرة في السوق

(ج) MIPS يحتوي على أكثر من 65 تعليمة، ويُعتبر لغة نموذجية (Template) لبقية المعالجات؛ فإذا فهمنا كيفية عمله سنفهم بسهولة كيفية عمل معظم المعالجات الأخرى

(د) أهميته تنحصر في كونه المعالج الوحيد القابل للاستخدام في الأجهزة المحمولة تحديداً

الشرح: يحتوي MIPS على أكثر من 65 تعليمة، ويُعتبر لغة نموذجية لبقية المعالجات، فإذا فهمنا كيف يعمل سنفهم بسهولة كيف تعمل معظم المعالجات الأخرى.

الأخطاء: الخيار الأول ينكر الانتشار الواسع الموثق لمIPS الموصوف قبل هذه النقطة مباشرة. الخيار الثاني والرابع يخترعان ادعاءات ضيقة (السعر، الحصرية للأجهزة المحمولة) غير مذكورة في التفسير الفعلي، الذي يركز على القيمة التعليمية/النموذجية لمIPS.

الجواب: ج.

س12: يحاول طالب تصنيف التعليمات lw و sw و add و and و branch و jump ضمن الفئات الخمس لعمليات MIPS. ما الفئات الخمس الصحيحة لهذه العمليات؟

(أ) عمليات الإدخال فقط، عمليات الإخراج فقط، عمليات المعالجة، عمليات التخزين، عمليات الشبكة

(ب) عمليات RISC، عمليات CISC، عمليات Stack، عمليات Accumulator، عمليات GPR

(ج) عمليات ثابتة الطول، عمليات متغيرة الطول، عمليات فورية، عمليات مؤجلة، عمليات متوازية

(د) العمليات الحسابية (Arithmetic مثل add, sub)، العمليات المنطقية (Logical مثل and, or)، نقل البيانات (Data Transfer مثل lw, sw)، التفرع الشرطي (Conditional Branch)، والقفز غير المشروط (Unconditional Jump)

الشرح: الفئات الخمس هي: العمليات الحسابية (add, sub, addi)، العمليات المنطقية (and, or, nor, andi)، نقل البيانات (lw لجلب البيانات من الذاكرة إلى السجل، sw لتخزين البيانات من السجل إلى الذاكرة)، التفرع الشرطي، والقفز غير المشروط.

الأخطاء: الخيارات الثلاثة الأخرى تخترع تصنيفات غير واردة إطلاقاً في القائمة الفعلية، وتخلط مفاهيم من سياقات أخرى تماماً (وحدات الإدخال/الإخراج، تصنيف RISC/CISC وأنماط تنظيم العمل، طول التعليمات) بدلاً من تصنيف أنواع العمليات المحدد المطلوب هنا.

الجواب: د.

س13: يلاحظ مهندس أن كل تعليمة MIPS تتكون من كلمة واحدة بطول ثابت مقداره 32 بت، تحتوي على كود التشغيل (OPcode) والمعاملات (Operands) بشكل منتظم، وأن المعالج يتجنب الوصول إلى الذاكرة كلما أمكن ذلك. أي قاعدة من قواعد MIPS المرتبطة بمبدأ RISC يصفها كل جزء من هذا السيناريو؟

(أ) الجزء الأول يصف قاعدة الحفاظ على انتظام التنسيق (كل تعليمة كلمة واحدة ثابتة الطول 32 بت تحتوي OPcode ومعاملات بشكل منتظم)، والجزء الثاني يصف قاعدة تقليل الوصول للذاكرة (استخدام السجلات كوسائط كلما أمكن ذلك)

(ب) كلا الجزأين يصفان القاعدة نفسها المتعلقة بتقليل عدد التعليمات الكلي في مجموعة التعليمات

(ج) الجزء الأول يصف قاعدة تقليل الوصول للذاكرة، والجزء الثاني يصف قاعدة الحفاظ على انتظام التنسيق، أي عكس الترتيب الصحيح

(د) لا علاقة لهذا السيناريو بأي من قواعد MIPS الثلاث؛ فهو يصف فقط بنية العتاد الفيزيائية

الشرح: يعتمد MIPS على ثلاث قواعد: (1) التعليمات تنفذ وظائف بسيطة، (2) الحفاظ على انتظام التنسيق: كل تعليمة كلمة واحدة ثابتة الطول 32 بت تحتوي OPcode ومعاملات بشكل منتظم، (3) تقليل الوصول للذاكرة: استخدام السجلات كوسائط كلما أمكن ذلك.

الأخطاء: الخيار الثاني يدمج قاعدتين منفصلتين تماماً (الانتظام والتقليل من الوصول للذاكرة) تحت وصف مختلف كلياً (تقليل عدد التعليمات). الخيار الثالث يبدل مطابقة كل جزء من السيناريو بالقاعدة الصحيحة. الخيار الرابع ينكر الصلة المباشرة رغم مطابقة السيناريو لقاعدتين من القواعد الثلاث حرفياً.

الجواب: أ.

س14: ثلاث تعليمات MIPS مختلفة: الأولى تستخدم السجلات فقط كمعاملات (مثل `add $t0,$s1,$s2`)، الثانية تستخدم سجلاً بالإضافة إلى رقم ثابت مثل عنوان أو عدد (مثل `lw $t0,4($s1)`)، والثالثة تُستخدم للانتقال إلى عنوان محدد في الكود. ما نوع كل تعليمة من هذه التعليمات الثلاث؟

(أ) الأولى J-type، الثانية R-type، الثالثة I-type

(ب) الأولى R-type (Register type)، معاملاتها سجلات فقط، الثانية I-type (Immediate type)، معاملاتها سجلات وأرقام ثابتة، الثالثة J-type (Jump type)، للقفز إلى عنوان محدد

(ج) الأولى I-type، الثانية J-type، الثالثة R-type

(د) الأنواع الثلاثة كلها متطابقة تماماً في طريقة تنسيق معاملاتها، ولا يوجد تصنيف فعلي بينها

الشرح: الأنواع الثلاثة: R-type (Register type) تستخدم السجلات فقط كمعاملات، I-type (Immediate type) معاملاتها سجلات وأرقام ثابتة (عناوين أو أعداد)، J-type (Jump type) تُستخدم للقفز إلى عنوان محدد في الكود.

الأخطاء: الخيار الأول والثالث يبدآن مطابقة كل تعليمة بنوعها الصحيح بطرق مختلفة. الخيار الرابع ينكر التصنيف الثلاثي الصريح بمعاملات مختلفة تماماً لكل نوع.

الجواب: ب.

س15: يتساءل طالب: لماذا تُصمَّم كل تعليمة جمع أو طرح في MIPS بحيث تحتوي دائماً على مصدرين ووجهة واحدة، بدلاً من السماح بعدد متغير من المعاملات حسب الحاجة؟ ما التفسير الصحيح لهذا الإلزام بحسب مبدأ Regularity Favors Simplicity؟

(أ) الإلزام ناتج عن قيود فيزيائية في عدد الأسلاك الكهربائية المتاحة داخل المعالج فقط، ولا علاقة له بتبسيط التصميم

(ب) الإلزام ناتج عن رغبة المصممين في تعقيد عملية البرمجة عمداً لتقليل عدد المبرمجين القادرين على استخدام MIPS

(ج) جعل جميع التعليمات بالشكل نفسه (عدد المعاملات نفسه) يجعل تصميم الـ Hardware أبسط، والانتظام يسهّل عملية ترميز (Encoding) التعليمات وتنفيذها بسرعة أعلى وتكلفة أقل

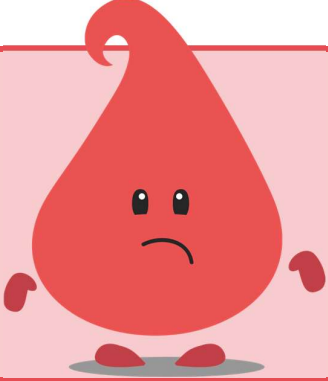
(د) الإلزام غير حقيقي؛ فيمكن استخدام أي عدد من المعاملات يشاء ضمن تعليمة واحدة في MIPS

الشرح: لماذا يجبرنا MIPS دائماً على استخدام 3 معاملات؟ لأن جعل جميع التعليمات بنفس الشكل (نفس عدد المعاملات) يجعل تصميم الـ Hardware أبسط، والانتظام يسهّل عملية ترميز التعليمات وتنفيذها بسرعة أعلى وتكلفة أقل.

الأخطاء: الخيار الأول يخترع قيداً فيزيائياً غير مذكور، متجاهلاً التفسير الفعلي المرتبط بتبسيط التصميم والترميز. الخيار الثاني يخترع دافعاً غير منطقي وغير مدعوم بأي إشارة. الخيار الرابع ينكر القاعدة الثابتة الموثقة صراحة والتي يُبنى عليها هذا الجزء بالكامل.

الجواب: ج.

س16: يريد مبرمج ترجمة العبارة $a = b + c + d + e$ إلى لغة MIPS Assembly، مع العلم أن كل تعليمة جمع في MIPS تقبل مصدرين فقط في كل مرة. كم سطر Assembly سيحتاجها هذا التعبير، وما الترتيب الصحيح لتنفيذها؟



(أ) سطر واحد فقط: Add a,b,c,d,e
 (ب) سطرين فقط: الأول يجمع b+c+d معاً، والثاني يضيف e للنتيجة
 (ج) أربعة أسطر، بواقع سطر منفصل لكل عملية جمع بين رقمين متتاليين من الأربعة
 (د) ثلاثة أسطر: Add a,b,c (لتصبح $a=b+c$) ثم Add a,a,d (لتصبح $a=(b+c)+d$) ثم Add a,a,e (لتصبح $a=(b+c)+d+e$)

الشرح: المثال المحلول: Add a,b,c # $a=b+c$ ، ثم Add a,a,d # $a=(b+c)+d$ ، ثم Add a,a,e # $a=(b+c)+d+e$. أي ثلاثة أسطر تستخدم فيها نتيجة كل خطوة كأحد مدخلات الخطوة التالية.

الأخطاء: الخيار الأول يخترع قدرة على قبول عدد غير محدود من المصادر في تعليمة واحدة، مناقضاً قاعدة المصدرين الثابتة التي تفرضها MIPS. الخيار الثاني يحاول جمع ثلاثة أرقام في سطر واحد، وهو أمر لا يدعمه تنسيق MIPS ثنائي المصدر. الخيار الثالث يبالغ في عدد الأسطر المطلوبة؛ فالحل الفعلي يستخدم إعادة توظيف الناتج التراكمي في السجل a، محتاجاً ثلاثة أسطر فقط لا أربعة.

الجواب: د.

س17: يسأل طالب عن عدد السجلات المتوفرة في بنية MIPS وحجم كل سجل منها. ما الإجابة الصحيحة؟

- (أ) يوجد 32 سجلاً في بنية MIPS، حجم كل سجل 32 بت (يُطلق على هذه الـ 32 بت اسم Word/كلمة)، وترقم السجلات من 0 إلى 31
 (ب) يوجد 64 سجلاً، حجم كل سجل 16 بت، وترقم من 1 إلى 64
 (ج) عدد السجلات غير ثابت ويختلف باختلاف البرنامج المُشغَّل على المعالج
 (د) يوجد 8 سجلات فقط، حجم كل سجل 64 بت، وترقم من 0 إلى 7

الشرح: العدد: يوجد 32 سجلاً في بنية MIPS. الوظيفة: تُستخدم لتخزين البيانات التي يتم الوصول إليها بشكل متكرر. الحجم: كل سجل حجمه 32 بت ويُطلق على هذه الـ 32 بت اسم Word. الترقيم: تُرقم من 0 إلى 31.

الأخطاء: الخيارات الثلاثة الأخرى تخترع أعداداً وأحجاماً وأنظمة ترقيم مختلفة تماماً عن المواصفات الدقيقة والصريحة (32 سجلاً، 32 بت لكل سجل، ترقيم من 0 إلى 31).

الجواب: أ.

س18: يستخدم مبرمج الاسم $t3\$$ في كود Assembly، ويريد معرفة الرقم الفعلي لهذا السجل الذي يفهمه المعالج (CPU) مباشرة. بحسب مخطط التقابل بين السجلات المؤقتة ($t0\$-t7\$$) وأرقامها (8-15)، ما رقم السجل $t3\$$ ؟

(أ) السجل رقم 3

(ب) السجل رقم 11

(ج) السجل رقم 16

(د) لا يمكن تحديد رقم السجل الفعلي

الشرح: السجلات المؤقتة $t0\$$ إلى $t7\$$ تقابل السجلات المرقمة من 8 إلى 15. بما أن $t0\$=8$ ، فإن $t3\$$ (وهو السجل الرابع في هذا النطاق، بإضافة 3 عن $t0\$$) يقابل الرقم $11=3+8$.

الأخطاء: الخيار الأول يفترض تطابقاً مباشراً بين الرقم الظاهر بالاسم ورقم السجل الفعلي، متجاهلاً الإضافة الثابتة (بداية من 8) المحددة صراحة. الخيار الثالث يخلط بين نطاق السجلات المؤقتة ($t\$$)، المقابل للأرقام (8-15) ونطاق السجلات المحفوظة ($s\$$)، المقابل للأرقام (16-23). الخيار الرابع ينكر جدول التقابل الثابت الصريح بين الأسماء وأرقام السجلات.

الجواب: ب.

س19: يشرح أستاذ لطلابه ثلاثة سجلات خاصة ضمن بنية MIPS، لكل منها وظيفة مختلفة عن السجلات العادية المستخدمة لتخزين المتغيرات. ما وظيفة كل من هذه السجلات الثلاثة؟

(أ) $zero\$$ يخزن أكبر قيمة عددية ممكنة، IR يحمل عنوان الذاكرة الحالي، PC يحمل نتيجة آخر عملية حسابية

(ب) $zero\$$ يُستخدم لتخزين متغيرات المستخدم المؤقتة، IR و PC كلاهما يشيران إلى نفس العنوان دائماً

(ج) $zero\$$ هو السجل رقم 0 وقيمته دائماً صفر (يُستخدم عند الحاجة لإجراء عمليات تتضمن الصفر)، IR (Instruction Register) هو السجل المسؤول عن حمل وتخزين التعليمات الحالية قيد التنفيذ، و PC (Program Counter) هو السجل الذي يحمل عنوان التعليمات التالية المقرر تنفيذها

(د) هذه السجلات الثلاثة غير موجودة فعلياً في بنية MIPS، وهي فقط مفاهيم نظرية عامة

الشرح: السجل الصفري (zero\$) بأنه السجل رقم 0 وقيمته دائماً صفر، يُستخدم عند الحاجة لإجراء عمليات تتضمن الصفر. سجل (IR (Instruction Register هو السجل المسؤول عن حمل وتخزين التعليمات التي يتم تنفيذها حالياً. سجل (PC (Program Counter هو السجل الذي يحمل عنوان التعليمات التالية المقرر تنفيذها.

الأخطاء: الخيار الأول والثاني يخترعان وظائف غير صحيحة تماماً لهذه السجلات الثلاثة، مناقضة للتعريفات الدقيقة. الخيار الرابع ينكر وجودها الفعلي رغم إدراجها صراحة ضمن السجلات ذات الوظائف الخاصة في MIPS تحديداً.

الجواب: ج.

س20: يقترح مهندس زيادة عدد السجلات في معالج MIPS من 32 إلى عدة مئات لتخزين المزيد من المتغيرات دفعة واحدة. ما الأثر السلبي المحتمل لهذا الاقتراح بحسب مبدأ Smaller is Faster؟

(أ) لا يوجد أي أثر سلبي محتمل

(ب) الأثر الوحيد المحتمل هو ارتفاع سعر تصنيع المعالج فقط

(ج) زيادة عدد السجلات تقلل زمن دورة الساعة (Clock Cycle Time) تلقائياً

(د) زيادة عدد السجلات تزيد مسافة الإشارة الكهربائية داخل المعالج، وتحتاج تعليمات البرنامج بتات أكثر لعنونة السجل

الشرح: هناك أثران محتملان لزيادة عدد السجلات: (1) زيادة زمن دورة الساعة، لأن زيادة عدد السجلات تؤدي إلى زيادة المسافة التي يجب أن تقطعها الإشارة الكهربائية داخل المعالج، وهذه المسافة الزائدة قد تؤدي إلى زيادة زمن دورة الساعة. (2) كلما زاد عدد السجلات احتجنا إلى عدد أكبر من البتات داخل التعليمات لتمثيل عنوان السجل، مما يجعل حجم التعليمات أكبر وأكثر تعقيداً.

الأخطاء: الخيار الأول ينكر وجود أي مقايضة، متجاهلاً وجود مبدأ تصميمي كامل مخصص لتفسير سبب عدم زيادة عدد السجلات تلقائياً. الخيار الثاني يختزل الأثر في التكلفة فقط، متجاهلاً الأثرين التقنيين المحددين (زمن دورة الساعة وحجم التعليمات) المفصلين. الخيار الثالث يذكر الاتجاه المعاكس تماماً؛ والصحيح أن زيادة المسافة قد تزيد (لا تقلل) زمن دورة الساعة.

الجواب: د.

س21: تُخصّص السجلات التالية للمتغيرات في العبارة $f \leftarrow s0: f = (g+h) - (i+j)$ ؛ (رقمه 16)، $g \leftarrow s1$ (رقمه 17)، $h \leftarrow s2$ (رقمه 18)، $i \leftarrow s3$ (رقمه 19)، $j \leftarrow s4$ (رقمه 20)، مع استخدام سجلين مؤقتين $t0$ (رقمه 8) و $t1$ (رقمه 9) للنتائج الوسيطة. ما التسلسل الصحيح لتعليمات MIPS Assembly (بالأسماء) اللازمة لتنفيذ هذه العبارة، وما مكافئها بالأرقام؟

- (أ) $add \$t0, \$s1, \$s2$ (تجمع $g+h$) ثم $add \$t1, \$s3, \$s4$ (تجمع $i+j$) ثم $sub \$s0, \$t0, \$t1$ (تطرح الناتجين وتُخزن في f)، ومكافئها بالأرقام: $add \$8, \$17, \$18$ ثم $add \$9, \$19, \$20$ ثم $sub \$16, \$8, \$9$
- (ب) $add \$s0, \$s1, \$s2$ ثم $add \$s0, \$s3, \$s4$ ثم $sub \$s0, \$s0, \$s0$ بحيث تُستخدم $s0$ لتخزين جميع النتائج الوسيطة دون الحاجة لسجلات مؤقتة
- (ج) $sub \$t0, \$s1, \$s2$ ثم $sub \$t1, \$s3, \$s4$ ثم $add \$s0, \$t0, \$t1$ أي عكس نوعي العمليتين (جمع بدل طرح وطرح بدل جمع)
- (د) لا يمكن ترجمة هذه العبارة إلى MIPS Assembly لأنها تحتوي على أكثر من عملية حسابية واحدة

الشرح: المثال المحلول: $add \$t0, \$s1, \$s2$ ثم $add \$t1, \$s3, \$s4$ ثم $sub \$s0, \$t0, \$t1$ ومكافئها بالأرقام: $add \$8, \$17, \$18$ ثم $add \$9, \$19, \$20$ ثم $sub \$16, \$8, \$9$ بعد تحديد أن $s0 = 16$ ، $s1 = 17$ ، $t0 = 8$ ، $t1 = 17$.

الأخطاء: الخيار الثاني يعيد الكتابة فوق $s0$ بشكل متكرر قبل إجراء عملية الطرح، مما يفقد القيم الوسيطة اللازمة، ويتجاهل الاستخدام الصحيح لسجلين مؤقتين ($t0$ ، $t1$) لهذا الغرض تحديداً. الخيار الثالث يبدّل نوعي العمليتين بالكامل (جمع بدل طرح والعكس)، منتجاً حساباً خاطئاً تماماً. الخيار الرابع ينكر قدرة MIPS على التعامل مع تعبيرات متعددة الخطوات، وهو ما يناقضه هذا المثال المحلول بالكامل.

الجواب: أ.

س22: يستخدم طالب المصطلحات الثلاثة **Architecture** و **Instruction Set** و **Instruction Set** بشكل متبادل معتقداً أنها مترادفة تماماً. ما التعريف الدقيق لكل مصطلح من هذه المصطلحات الثلاثة؟

- (أ) الثلاثة مترادفة تماماً وتشير للمفهوم نفسه دون أي فرق
- (ب) **Instruction Set** هي كلمات لغة الحاسوب، **Instruction Set** هي مجموعة الأوامر التي تفهمها بنية الحاسوب، و **Architecture** هي مجموعة الدارات حيث كل دارة تستقبل مجموعة تعليمات
- (ج) **Instruction Set** هي مجموعة الدارات، **Instruction Set** هي كلمات اللغة، و **Architecture** هي الأوامر التي تفهمها البنية
- (د) المصطلحات الثلاثة تخص فقط لغات البرمجة عالية المستوى، ولا علاقة لها بمعالج الحاسوب

الشرح: Instruction هي كلمات لغة الحاسوب، Instruction Set هي مجموعة أوامر تفهمها بنية الحاسوب، Architecture هي عبارة عن مجموعة الدارات، كل دارة تستقبل مجموعة تعليمات.

الأخطاء: الخيار الأول ينكر التمييز الثلاثي الصريح المطروح في افتتاحية هذا الجزء. الخيار الثالث يبدل تعريفات المصطلحات الثلاثة بشكل كامل مقارنة بالتعريفات الفعلية. الخيار الرابع يحصر هذه المفاهيم بلغات البرمجة عالية المستوى، بينما تُعرّف صراحة كمفاهيم على مستوى المعالج/العتاد.

الجواب: ب.

المحاضرة 4: تعليمات معالج MIPS

س1: تعليمية MIPS من نوع R-type تحتوي على الحقول: op, rs, rt, rd, shamt, funct. ما حجم كل حقل من هذه الحقول بالبت (bit)؟

أ) 6 bit لكل من op و funct، و 5 bit لكل من rs و rt و rd و shamt

ب) 5 bit لكل من op و funct، و 6 bit لكل من rs و rt و rd و shamt

ج) 16 bit لجميع الحقول الستة بالتساوي

د) 6 bit لجميع الحقول الستة بالتساوي

الشرح: الحقول الفعلية وأحجامها: op(6bit), rs(5bit), rt(5bit), rd(5bit), shamt(5bit), funct(6bit).

الأخطاء: الخيار B يعكس التوزيع الصحيح للأحجام بين op/funct والحقول الأخرى. الخيار C و D يخترعان توزيعاً موحداً غير متطابق مع البنية الفعلية المختلطة بين حقول 6 بت وحقول 5 بت.

الجواب: أ.

س2: يتساءل طالب: لماذا يعتمد معالج MIPS على حقل funct لتحديد نوع العملية الدقيقة (كالجمع أو الطرح) في تعليمات type-R، بدلاً من الاعتماد على حقل opcode مباشرة كما في type-I و type-J؟ ما السبب؟

(أ) لأن حقل opcode غير موجود إطلاقاً ضمن تنسيق type-R

(ب) لأن حقل opcode يكون دائماً صفراً في جميع تعليمات type-R، فلا يكفي وحده للتمييز بين العمليات المختلفة التي تشترك بهذا الصفر

(ج) لأن عدد تعليمات type-R قليل جداً بحيث لا حاجة لحقل إضافي للتمييز بينها

(د) لأن حقل funct أكبر حجماً من opcode ويمكنه استيعاب عدد أكبر من الرموز الثنائية

الشرح: تعليمات type-R كثيرة العدد وجميعها تشترك بـ $opcode=0$ ، فاستُخدم حقل funct للتمييز بينها. أما type-I و type-J فعدد تعليماتهما أقل وتوجد مساحة كافية في opcode لتمييزها مباشرة.

الأخطاء: الخيار A ينكر وجود حقل opcode في type-R رغم أنه موجود ويساوي صفراً دائماً. الخيار C يعكس الحقيقة؛ فتعليمات type-R كثيرة وليست قليلة. الخيار D يخترع فرق حجم غير صحيح؛ فكل الحقلين 6 بت بالحجم نفسه.

الجواب: ب.

س3: تعليمة MIPS من نوع I-type مثل $lw \$t0, 32(\$s3)$. ما تنسيق حقول هذه التعليمة بالترتيب؟

(أ) $op(6bit) - rs(5bit) - rt(5bit) - rd(5bit) - shamt(5bit) - funct(6bit)$

(ب) $op(6bit) - rs(5bit) - rt(5bit) - Immediate/Address(16bit)$

(ج) $op(6bit) - Address(26bit)$

(د) $op(16bit) - rs(8bit) - rt(8bit)$

الشرح: تنسيق I-type: $op(6bit)$, $rs(5bit)$, $rt(5bit)$, $Immediate/Address(16bit)$.

الأخطاء: الخيار A يصف تنسيق type-R بدلاً من type-I. الخيار C يصف تنسيق type-J. الخيار D يخترع أحجام حقول غير موجودة في أي تنسيق فعلي بـ MIPS.

الجواب: ب.

س4: التعليمة 1000 ز تنتمي إلى نوع J-type وتستخدم للقفز غير المشروط إلى عنوان محدد في البرنامج. ما تنسيق حقول هذا النوع؟

(أ) $op(6bit) - Address(26bit)$

(ب) $op(6bit) - rs(5bit) - rt(5bit) - Immediate(16bit)$

(ج) $op(6bit) - rs(5bit) - rt(5bit) - rd(5bit) - shamt(5bit) - funct(6bit)$

(د) $op(26bit) - Address(6bit)$

الشرح: تنسيق J-type: $op(6bit) + Address(26bit)$.

الأخطاء: الخيار A يصف تنسيق type-I. الخيار B يصف تنسيق type-R. الخيار C يعكس توزيع البتات الصحيح؛ فالعنوان هو من يشغل 26 بت وليس opcode.

الجواب: أ.

س5: حقل opcode في أي تعليمة MIPS يتكون من 6 بتات. ما العدد الأقصى للعمليات (Operations) المختلفة التي يمكن تمثيلها عبر هذا الحقل؟

(أ) 32 عملية (2^5)

(ب) 64 عملية (2^6)

(ج) 6 عمليات فقط، بعدد البتات نفسه

(د) 128 عملية (2^7)

الشرح: عدد العمليات الممكنة $= 2^6 = 64$.

الأخطاء: الخيار A يستخدم أس 5 بدلاً من 6. الخيار C يخلط بين عدد البتات نفسه وعدد التمثيلات الممكنة عبرها. الخيار D يستخدم أس 7 بدلاً من 6.

الجواب: ب.

س6: تعليمة $\text{add } \$t1, \$s1, \$s2$ حيث القيم العددية للسجلات: $t1=9$, $s1=17$, $s2=18$. ما القيم العددية الصحيحة لحقول (op, rs, rt, rd) بعد ترميز هذه التعليمة بتنسيق type-R؟



(أ) $op=32, rs=9, rt=17, rd=18$

(ب) $op=0, rs=9, rt=17, rd=18$

(ج) $op=0, rs=17, rt=18, rd=9$

(د) $op=0, rs=18, rt=17, rd=9$

الشرح: rs = المصدر الأول = $s1=17$, rt = المصدر الثاني = $s2=18$, rd = الوجهة = $t1=9$ و $opcode=0$ دائماً في type-R (بينما $func=32$ يحدد أنها عملية جمع تحديداً).

الأخطاء: الخيار A يضع القيمة 32 (وهي قيمة funct الخاصة بالجمع) في حقل op الذي يجب أن يبقى صفراً دائماً في type-R. الخيار C يضع سجل الوجهة 9 في موضع rs مما يزيح باقي القيم خطأً. الخيار D يبدّل قيمتي rs و rt (17 و 18) عن موضعيهما الصحيحين.

الجواب: ج.

س7: يقترح مهندس استخدام تنسيق type-R نفسه (بحقول bit-5 لكل معامل) لتعليمات type-I أيضاً بدلاً من تصميم تنسيق منفصل لها. لماذا يُعتبر هذا الاقتراح غير عملي؟

(أ) لأن تعليمات type-I لا تحتاج أي حقل مخصص للثوابت أو العناوين إطلاقاً

(ب) لأن تنسيق type-R لا يحتوي على حقل opcode، وهو ضروري لكل تعليمة

(ج) لأن حقل bit-5 يسمح فقط بنطاق قيم صغير جداً (تقريباً [16 إلى 15])، بينما تحتاج تعليمات type-I لتمثيل ثوابت وعناوين أكبر بكثير، وهذا ما دفع لمبدأ Good design demands good compromises

(د) لأن عدد بتات type-R أكبر من 32 بت، بينما كل تعليمة MIPS يجب أن تكون 32 بت بالضبط

الشرح: حقل bit-5 يمثل كحد أقصى $2^5=32$ ، أي نطاقاً محدوداً جداً لا يكفي لتمثيل ثوابت وعناوين type-I، وهذا التعارض بين طول التعليمة الثابت والحاجة للمرونة أدى لمبدأ Good design demands good compromises، أي استخدام تنسيقات مختلفة حسب نوع التعليمة.

الأخطاء: الخيار A ينكر حاجة معروفة وأساسية لتعليمات مثل `addi lw`. الخيار B يخطئ بادعاء غياب حقل `opcode` في `type-R`، وهو موجود (6 بت، قيمته صفر). الخيار D يخترع اختلافاً في الطول الكلي؛ فكلتا التنسيقين 32 بت بالضبط، وهذا ليس مصدر المشكلة.

الجواب: ج.

س8: التعليمة $LW \$t0, 4(\$t1)$. ماذا تنفذ هذه التعليمة بالضبط؟

- (أ) تجلب القيمة من $t1\$$ وتضعها في العنوان $(t0 + 4\$)$
- (ب) تخرّن القيمة الموجودة في $t0\$$ في العنوان $(t1 + 4\$)$
- (ج) تجلب القيمة الموجودة في العنوان $(t1 + 4\$)$ وتضعها في $t0\$$
- (د) تجمع القيمتين $t0\$$ و $t1\$$ ثم تضيف 4 للناتج

الشرح: تجلب القيمة الموجودة في العنوان $(t1+4\$)$ وتضعها في $t0\$$ ، حيث يُستخدم عنوان $t1\$$ (وليس قيمته) كأساس لحساب العنوان.

الأخطاء: الخيار A يعكس المصدر والوجهة. الخيار B يصف سلوك تعليمة `sw` وليس `lw`. الخيار C يخترع عملية حسابية لا علاقة لها بدلالة تعليمات نقل البيانات.

الجواب: ج.

س9: التعليمة $SW \$t0, 16(\$t1)$. ماذا تنفذ هذه التعليمة بالضبط؟

- (أ) تخرّن قيمة $t0\$$ في العنوان $(t1 + 16\$)$
- (ب) تجلب القيمة من العنوان $(t1 + 16\$)$ وتضعها في $t0\$$
- (ج) تخرّن قيمة $t1\$$ في العنوان $(t0 + 16\$)$
- (د) تبادل القيمتين بين $t0\$$ و $t1\$$ عبر الإزاحة 16

الشرح: تخزن قيمة $t0\$$ في العنوان $(t1+16\$)$.

الأخطاء: الخيار B يصف سلوك lw وليس sw . الخيار C يعكس المصدر والوجهة. الخيار D يخترع عملية تبادل غير موجودة في دلالة sw .

الجواب: أ.

س10: تعتمد بنية MIPS على Byte Addressing، حيث تشغل الكلمة الواحدة 32 (Word) بت. كم عنوان (Byte Address) تشغله الكلمة الواحدة في الذاكرة، وما الفارق بين عناوين كلمتين متتاليتين؟

- أ) تشغل الكلمة عنواناً واحداً فقط (بايت واحد)، والفارق بين كلمتين متتاليتين هو 1
- ب) تشغل الكلمة 4 عناوين (4 بايت)، والفارق بين عناوين كلمتين متتاليتين هو 4
- ج) تشغل الكلمة 32 عنواناً (بايت واحد لكل بت)، والفارق بين كلمتين متتاليتين هو 32
- د) تشغل الكلمة 8 عناوين، والفارق بين كلمتين متتاليتين هو 8

الشرح: الكلمة = $32 \text{ byte} = 4 \text{ bit}$. فتشغل 4 عناوين، والفارق بين عناوين كلمتين متتاليتين = 4 (مثال: 0، 4، 8، 12...).

الأخطاء: الخيار A يخلط بين دقة العنوان (عنوان واحد = بايت واحد) وحجم الكلمة نفسها. الخيار C يعامل كل بت على أنه بحاجة لعنوان مستقل. الخيار D يخترع حجم كلمة (8 عناوين) لا يطابق حجمها الفعلي (4 بايت).

الجواب: ب.

س11: لتحميل قيمة العنصر A[4] في السجل $t0\$$ ، حيث السجل $t2\$$ يمثل العنوان الأساسي للمصفوفة A. ما التعليمات الصحيحة وما قيمة الإزاحة (Offset) المستخدمة؟

- أ) $lw \$t0, 4(\$t2)$ — الإزاحة = 4 مباشرة دون ضرب
- ب) $lw \$t0, 8(\$t2)$ — الإزاحة = $4 + 4$
- ج) $lw \$t0, 16(\$t2)$ — الإزاحة = $4 \times 4 = 16$
- د) $lw \$t0, 20(\$t2)$ — الإزاحة = $4 \times (1 + 4)$

الشرح: الإزاحة = الفهرس $\times 4 = 4 \times 4 = 16$.

الأخطاء: الخيار A ينسى ضرب الفهرس بحجم الكلمة (4)، ويستخدم الفهرس مباشرة كإزاحة. الخيار B يستخدم عملية جمع بدلاً من الضرب. الخيار D يضيف 1 زائداً للفهرس قبل الضرب بشكل خاطئ.

الجواب: ج.

س12: لدينا التعليمة بلغة $A[12] = h + A[8]C$ ، حيث h موجودة في $\$2$ ، والعنوان الأساسي للمصفوفة A في $\$3$. ما التسلسل الصحيح لتعليمات MIPS المكافئة؟

أ) $(lw \$t0, 32(\$s3 \text{ ثم } add \$t0, \$s2, \$t0 \text{ ثم } sw \$t0, 48(\$s3$

ب) $(lw \$t0, 8(\$s3 \text{ ثم } add \$t0, \$s2, \$t0 \text{ ثم } sw \$t0, 12(\$s3$

ج) $(lw \$t0, 48(\$s3 \text{ ثم } add \$t0, \$s2, \$t0 \text{ ثم } sw \$t0, 32(\$s3$

د) $(sw \$t0, 48(\$t0 \text{ ثم } lw \$t0, 32(\$t0 \text{ ثم } add \$t0, \$s2, \$s3$

الشرح: إزاحة $A[8] = 8 \times 4 = 32$ ، وإزاحة $A[12] = 12 \times 4 = 48$: نحمل $A[8]$ بالإزاحة 32، نجمعها مع h ، ثم نخزن الناتج عند إزاحة $A[12]$ وهي 48.

الأخطاء: الخيار A يستخدم الفهرسين (8، 12) مباشرة كإزاحتين دون ضربهما بحجم الكلمة (4). الخيار B يبدل إزاحتي التحميل والتخزين (يستخدم إزاحة $A[12]$ للتحميل وإزاحة $A[8]$ للتخزين، معكوستين). الخيار C يخترع ترتيب عمليات خاطئاً (جمع العنوان الأساسي مع h قبل التحميل، بينما التسلسل الصحيح هو تحميل ثم جمع ثم تخزين).

الجواب: أ.

You will finish soon ...

س13: القيمة $x234567890$ مُخزّنة في الذاكرة ضمن كلمة واحدة. كيف يختلف ترتيب تخزين البايتات بين نظامي Little-endian و Big-endian؟

- (أ) في Big-endian يُخزّن البايت الأقل أهمية في العنوان الأصغر، وفي Little-endian يُخزّن البايت الأكثر أهمية في العنوان الأصغر (عكس التسمية تماماً)
- (ب) لا يوجد فرق فعلي بين النظامين؛ كلاهما يخزن البايتات بالترتيب نفسه
- (ج) Big-endian يُستخدم فقط مع الأعداد الموجبة، و Little-endian يُستخدم فقط مع الأعداد السالبة
- (د) في Big-endian يُخزّن البايت الأكثر أهمية (MSB) في العنوان الأصغر، بينما في Little-endian يُخزّن البايت الأقل أهمية (LSB) في العنوان الأصغر

الشرح: Big-endian: يبدأ ترقيم البايتات من الطرف الأكبر، ويُخزّن MSB في العنوان الأصغر. Little-endian: يبدأ الترقيم من الطرف الأصغر، ويُخزّن LSB في العنوان الأصغر. عنوان الكلمة نفسه لا يتغير بين النظامين؛ ترتيب البايتات هو ما يختلف.

الأخطاء: الخيار A يعكس تعريف النظامين بالكامل. الخيار B ينكر الفرق الموثق بمثال كامل. الخيار C يخترع علاقة بالإشارة (موجب/سالب) لا صلة لها إطلاقاً بمفهوم endianness.

الجواب: د.

س14: بفرض $0x234567890 = s_0$ ، وتم تنفيذ: $s_0, 0 \text{ } \$s_0 \text{ } sw \text{ } (0\$)$ ثم $s_0, 1 \text{ } \$s_0 \text{ } lb \text{ } (0\$)$ في نظام Big-endian. ما القيمة الجديدة لـ s_0 بعد تنفيذ هذين السطرين؟

- (أ) $x000000230$
- (ب) $x000000890$
- (ج) $x234567890$ دون أي تغيير
- (د) $x000000450$

الشرح: في Big-endian يُخزّن MSB (23) عند العنوان الأصغر (0)، فتصبح البايتات: العنوان $23=0$ ، العنوان $45=1$ ، العنوان $67=2$ ، العنوان $89=3$. تعليمة lb تجلب البايت عند العنوان $1=(1+0)$ وهو 45، وبما أنه موجب ($x450$) فإن التوسيع (sign extension) يعطي $x000000450$.

الأخطاء: الخيار A يقرأ البايت عند العنوان 0 (وهو 23) بدلاً من العنوان 1 المحدد فعلياً في التعليمية. الخيار B يقرأ من موضع بايت خاطئ آخر. الخيار C يفترض خطأً أن ba تُحمل الكلمة كاملة بدلاً من بايت واحد محدد بعنوانه.

الجواب: د.

س15: بدلاً من تحميل الثابت (4) إلى سجل مؤقت أولاً ثم جمعه، يوفر MIPS تعليمية الجمع الفوري addi للتعامل مع الثوابت الصغيرة مباشرة. ما الفائدة الأساسية لاستخدام addi بدلاً من تحميل الثابت عبر lw ثم الجمع؟



- (أ) تجنّب الوصول إلى الذاكرة، مما يجعل التنفيذ أسرع ويستهلك طاقة أقل
- (ب) السماح بتمثيل ثوابت أكبر من تلك التي يمكن تمثيلها بتعليمية lw
- (ج) استبدال الحاجة لوجود سجلات مؤقتة بشكل كامل في جميع عمليات الجمع
- (د) تحويل عملية الجمع إلى عملية طرح تلقائياً دون تعليمية Subi منفصلة

الشرح: addi لا يحتاج وصولاً للذاكرة، وبالتالي أسرع بكثير ويستهلك طاقة أقل مقارنة بتحميل الثابت أولاً ثم جمعه.

الأخطاء: الخيار A يخترع ميزة سعة غير مذكورة في التفسير الفعلي. الخيار B مبالغ فيه؛ addi يتجنب خطوة تحميل واحدة للثوابت، لا يلغي الحاجة للسجلات المؤقتة عموماً. الخيار C يخلط بين addi ونقطة منفصلة تماماً (عدم وجود subi).

الجواب: أ.

س16: يبحث مبرمج عن تعليمية subi لطرح ثابت من سجل مباشرة في MIPS. ما الحقيقة بخصوص هذه التعليمية؟

- (أ) توجد تعليمية Subi وتعمل بشكل مطابق تماماً لـ addi لكن بإشارة معكوسة داخلياً
- (ب) Subi موجودة لكنها تتطلب الوصول للذاكرة بعكس addi
- (ج) Subi موجودة فقط ضمن تعليمات type-R وليس type-I
- (د) لا توجد تعليمية Subi في MIPS؛ يُستخدم بدلاً منها addi مع قيمة ثابتة سالبة

الشرح: لا توجد تعليمة Subi، ويُستخدم بدلاً منها addi مع ثابت سالب.

الأخطاء: الخيارات B و C تخترع خصائص لتعليمة غير موجودة أصلاً، وتناقض النص الصريح الذي ينفي وجودها تماماً. الخيار A يخترع تعليمة وهمية بخصائص غير حقيقية.

الجواب: د.

س17: يُستخدم عدد كامل من 32 بت لتمثيل قيمة غير موقعة (Unsigned) في الذاكرة. ما المدى الكامل للقيم التي يمكن تمثيلها؟

(أ) من 0 إلى $2^{32}-1$ (أي من 0 إلى 4,294,967,295)

(ب) من $2^{31}-1$ إلى 2^{31}

(ج) من 0 إلى $2^{16}-1$ فقط

(د) من 1 إلى 2^{32} بدون قيمة صفر

الشرح: المدى غير الموقَّع = من 0 إلى $2^{32}-1$ = من 0 إلى 4,294,967,295.

الأخطاء: الخيار A يصف المدى الموقَّع (Two's Complement) وليس غير الموقَّع. الخيار B يستخدم مدى 16 بت غير ذي علاقة. الخيار C يستبعد الصفر ويزيح المدى بشكل غير صحيح.

الجواب: أ.

س18: في تمثيل Sign and Magnitude، يُفصل بت الإشارة (المرتبة الأكثر أهمية) عن باقي البتات التي تمثل القيمة المطلقة للعدد. ما المشكلة الرئيسية الموثقة لهذا التمثيل؟

(أ) عدم القدرة على تمثيل الأعداد السالبة إطلاقاً بهذا النظام

(ب) وجود تمثيلين مختلفين للصفر ($0+$ و $0-$)، إضافة لتعقيد إضافي في تصميم الدارات بسبب معالجة بت الإشارة بشكل منفصل

(ج) الحاجة لعدد بتات أكبر من الأنظمة الأخرى لتمثيل العدد نفسه

(د) عدم توافق هذا النظام مع عمليات الجمع الثنائي بأي شكل

الشرح: المشكلتان الموثقتان: وجود تمثيلين للصفر ($0+$ و $0-$), والحاجة لمعالجة خاصة لبت الإشارة مما يزيد تعقيد تصميم الدارات.

الأخطاء: الخيار A خاطئ؛ هذا النظام مصمم أصلاً لتمثيل الأعداد السالبة، والمشكلة في كيفية تمثيلها لا في استحالة ذلك. الخيار C يخترع فرق حجم غير مذكور. الخيار D مبالغ فيه؛ لم يُذكر عدم توافق كامل مع الجمع، بل مشكلتان محددتان فقط.

الجواب: ب.

س19: من بين ثلاثة أنظمة لتمثيل الأعداد الموقعة (Two's Complement، One's Complement، Sign and Magnitude)، أي نظام هو المعتمد فعلياً في الأنظمة الحاسوبية الحديثة؟

أ) One's Complement

ب) Sign and Magnitude

ج) Two's Complement

د) لا يوجد نظام معتمد؛ تختار كل شركة تصنيع نظامها الخاص بحرية تامة

الشرح: Two's Complement هو الطريقة المعتمدة لتمثيل الأعداد الموقعة في الأنظمة الحاسوبية، إذ يمثل العدد بالكامل بشكل ثنائي واحد دون فصل الإشارة عن القيمة، متجاوزاً مشاكل النظامين الآخرين.

الأخطاء: الخيار A و B ينسبان صفة الاعتماد الفعلي لنظامين موثق أن لديهما مشكلة تكرار تمثيل الصفر. الخيار D ينكر وجود معيار صناعي فعلي موثق بوضوح.

الجواب: ج.

س20: يريد مهندس تمثيل العدد -7 باستخدام Two's Complement، بدءاً من التمثيل الثنائي للقيمة المطلقة 7 (0111). ما الخطوات الصحيحة والمتسلسلة لهذا التحويل؟

- (أ) كتابة القيمة المطلقة ثنائياً، ثم إضافة 1 مباشرة دون قلب أي بت
- (ب) قلب بت الإشارة فقط (المرتبة الأكثر أهمية) دون التعرض لباقي البتات
- (ج) كتابة القيمة المطلقة ثنائياً، ثم قلب البتات، دون أي إضافة لاحقة
- (د) كتابة القيمة المطلقة ثنائياً، ثم قلب جميع البتات ($1 \leftrightarrow 0$)، ثم إضافة 1 للناتج

الشرح: الخطوات الثلاث: كتابة القيمة المطلقة ثنائياً، قلب جميع البتات ($1 \leftrightarrow 0$)، ثم إضافة 1 للناتج.

الأخطاء: الخيار A يتخطى خطوة قلب البتات الأساسية. الخيار B يقلب بتاً واحداً فقط (وهو أسلوب Sign and Magnitude المعيب، لا Two's Complement). الخيار C يتوقف عند قلب البتات دون الخطوة الأخيرة الضرورية (إضافة 1)، وهذا يطابق One's Complement بدلاً من Two's Complement.

الجواب: د.

س21: يريد مبرمج توسيع عدد ممثّل بـ 16 بت إلى تمثيل مكافئ بـ 32 بت، سواء كان العدد موجباً أم سالباً. ما القاعدة الصحيحة للتوسيع (Sign Extension)؟

- (أ) إضافة أصفار دائماً بغض النظر عن إشارة العدد الأصلي
- (ب) إضافة واحدات دائماً بغض النظر عن إشارة العدد الأصلي
- (ج) تكرار البت الأقل أهمية (LSB) في البتات الجديدة المضافة
- (د) تكرار البت الأكثر أهمية (MSB) في البتات الجديدة المضافة: أصفار إذا كان العدد موجباً ($MSB=0$)، وواحدات إذا كان سالباً ($MSB=1$)

الشرح: يُكرّر بت الإشارة (MSB) عند التوسيع: أصفار إذا كان العدد موجباً، وواحدات إذا كان سالباً، مما يحافظ على القيمة نفسها بعد زيادة عدد الخانات.

الأخطاء: الخيار A و B يطبقان قاعدة ثابتة واحدة بغض النظر عن إشارة العدد، بينما المثالان المحلولان (+2 و -2) يوضحان تمييزاً واضحاً بين الحالتين. الخيار C يخلط بين MSB (الذي يحدد نمط التوسيع) و LSB (غير ذي صلة بعملية التوسيع).

الجواب: د.

س22: عند تنفيذ تعليمة الضرب \$s2, \$s3 mult في MIPS، حيث قد يكون ناتج ضرب رقمين من 32 بت أكبر من سعة سجل واحد. أين تُخزن نتيجة هذه العملية؟

- (أ) في السجل الأول register1 مباشرة بغض النظر عن حجم الناتج
- (ب) في سجلين خاصين: HI يحمل الـ 32 بت العليا من الناتج، و LO يحمل الـ 32 بت الدنيا
- (ج) في الذاكرة الرئيسية RAM حصراً
- (د) يُقْتَطَع الناتج تلقائياً إلى 32 بت الأدنى فقط، وتُهمَل الـ 32 بت العليا

الشرح: تُخزن النتيجة في سجلين خاصين: HI يحمل الـ 32 بت العليا، و LO يحمل الـ 32 بت الدنيا، لتفادي فقدان أي بت من الناتج الذي قد يصل حجمه إلى 64 بت.

الأخطاء: الخيار A يتجاهل التصميم الفعلي المعتمد على سجلين مخصصين لا سجل عادي واحد. الخيار C يخترع موقع تخزين غير مذكور إطلاقاً في هذا السياق. الخيار D يصف فقداناً للبيانات يناقض الغرض الأساسي من استخدام سجلين، وهو الحفاظ على الناتج كاملاً دون فقدان أي بت.

الجواب: ب.

س23: تعليمة andi \$t0, \$t0, 0xFFFF. ما الأثر الناتج عن هذه العملية على بتات السجل؟

- (أ) تُحوّل جميع بتات السجل إلى واحدات بغض النظر عن قيمة القناع
- (ب) تبقى جميع بتات السجل كما هي دون أي تغيير مهما كانت قيمة القناع
- (ج) تُصَفّر (تصبح 0) البتات المقابلة للأصفار في القناع، بينما تبقى البتات المقابلة للواحدات (F) كما هي دون تغيير
- (د) يُصَفّر السجل بالكامل إلى القيمة 000000000x دائماً

الشرح: منطق AND: أي بت يُقارن بصفر في القناع تكون نتيجته صفراً دائماً (يُحجب)، وأي بت يُقارن بواحد في القناع تبقى قيمته الأصلية دون تغيير.

الأخطاء: الخيار A يصف منطق OR وليس AND. الخيار B ينكر الأثر الفعلي لعملية الإخفاء (Masking) بالكامل. الخيار D يبالغ في الأثر؛ فقط البتات المقابلة للأصفر في القناع تُصَفَّر، لا السجل كاملاً بالضرورة.

الجواب: ج.

س24: لدينا $t0=0x12345678$ ، وننفذ: $ori\ \$t0,\$t0,0xFFFF$. ما القيمة الجديدة لـ $t0$ بعد هذه العملية؟



أ) $0xFFFFFFFF$

ب) $0x123456780$

ج) $0x000056780$

د) $0x1234FFFF0$

الشرح: القناع $0xFFFF0$ يعني أصفراً في الـ 16 بت العليا وواحدات في الـ 16 بت الدنيا. البتات العليا (1234) تُقارَن بأصفر فتبقى كما هي، والبتات الدنيا (5678) تُقارَن بواحدات فتتحول جميعها لواحدات (FFFF)، فتصبح النتيجة $0x1234FFFF0$.

الأخطاء: الخيار A يبالغ في الأثر ليشمل السجل كاملاً بدلاً من الجزء المقابل للواحدات في القناع فقط. الخيار B ينكر الأثر الفعلي لعملية OR مع قناع من الواحدات. الخيار C يخترع نتيجة تصفير لا تطابق منطق عملية OR إطلاقاً (OR لا يُصَفِّر البتات أبداً).

الجواب: د.

المحاضرة 5: التعليمات المنطقية، الإجراءات، وأنماط العنوان في MIPS

س1: التعليمات 4, \$s0, \$t2, \$t2, \$s0 من نوع type-R، وتُستخدم لإزاحة منطقية نحو اليسار. ما العملية الحسابية المكافئة لهذه التعليمات؟

(أ) $t2 = \$s0 \times 2^4$ (أي $t2 = \$s0 \times 16$)

(ب) $t2 = \$s0 \div 2^4$ (أي $t2 = \$s0 \div 16$)

(ج) $t2 = \$s0 + 4$

(د) $t2 = \$s0 \bmod 4$

الشرح: الإزاحة نحو اليسار بمقدار i تعادل الضرب بـ 2^i ، وهذا مطابق لما ورد: `sll $t2,$s0,4` يكافئ `.mul $t2,$s0,16`.

الأخطاء: الخيار B يصف الإزاحة نحو اليمين (قسمة) وليس اليسار. الخيار C وD يخترعان عمليتين غير متعلقتين بمنطق الإزاحة إطلاقاً.

الجواب: أ.

س2: في صيغة `sll rd, rt, shamt` (وبشكل عام في تعليمات الإزاحة من نوع type-R)، يبقى حقل `rs` غير مستخدم فعلياً. ما القيمة التي يحملها حقل `rs` في هذه الحالة، وماذا يمثل حقل `shamt`؟

(أ) `rs` يحمل عنوان الذاكرة الهدف، و `shamt` يمثل رقم السجل المصدر

(ب) `rs` يبقى بلا استخدام فعلي، بينما 5 `shamt` (بت) يمثل مقدار الإزاحة (shift amount)

(ج) `rs` يحمل قيمة الإزاحة نفسها، و `shamt` يمثل السجل الهدف

(د) كلا الحقلين غير مستخدمين، والإزاحة تُحدّد حصراً من حقل `funct`

الشرح: `shamt` هي اختصار لـ shift amount، وتأخذ 5 بت لتمثيل مقدار الإزاحة، بينما حقل `rs` لا يُستخدم فعلياً في تعليمات الإزاحة.

الأخطاء: الخيار A و C يبدّلان الأدوار الصحيحة بين الحقليين. الخيار D ينكر الوظيفة الفعلية لحقل shamit الموضحة صراحة.

الجواب: ب.

س3: تعلّيمتا SRL (Logical Right Shift) و SRA (Arithmetic Right Shift) تُنفذان إزاحة نحو اليمين، لكن تختلفان في كيفية ملء البتات الفارغة الناتجة عن الإزاحة. ما الفرق بينهما؟

- (أ) SRL تملأ الفراغات ببتات الإشارة (البت 31)، و SRA تملأها بأصفار دائماً
- (ب) كلاهما يملأ الفراغات بأصفار دائماً بغض النظر عن نوع التعلّيم
- (ج) SRL تملأ الفراغات بأصفار دائماً، بينما SRA تملأ الفراغات بتكرار بت الإشارة (البت 31) للحفاظ على إشارة العدد
- (د) كلاهما يملأ الفراغات بواحدات دائماً بغض النظر عن نوع التعلّيم

الشرح: SRL تملأ الفراغات بأصفار (Logical)، بينما SRA (Arithmetic) تملأ الفراغات بتكرار بت الإشارة (البت 31) للحفاظ على إشارة العدد الأصلي.

الأخطاء: الخيار A يعكس السلوكين. الخيار B و D يطبقان قاعدة ثابتة واحدة على كلا التعلّيمتين، متجاهلين الفرق الموثق بينهما بمثال محلول.

الجواب: ج.

س4: لا توجد تعلّيم NOT مباشرة، ولا توجد تعلّيم NOR، ضمن مجموعة تعلّمات MIPS. كيف يمكن الحصول على معاكس (النفي المنطقي) لقيمة سجل معين باستخدام تعلّمات MIPS المتوفرة؟

- (أ) باستخدام تعلّيم sra مع قيمة إزاحة كافية لعكس جميع البتات
- (ب) لا يمكن الحصول على معاكس القيمة إطلاقاً في MIPS دون كتابة دائرة عتادية إضافية
- (ج) باستخدام تعلّيم andi مع القناع 0xFFFFFFFF
- (د) باستخدام تعلّيم nor مع السجل \$zero كأحد المعاملين

الشرح: $\$t0, \$zero, \$t2$ nor يعطي معاكس $\$t0$ ، لأن $(a \text{ or } 0)$ منفيًا يساوي $(\text{not } a)$ ، ولما أن $\$zero$ يحفظ دائماً القيمة صفر فإن هذه الطريقة تحقق النفي المنطقي.

الأخطاء: الخيار A يخترع استخداماً غير صحيح لتعليمة الإزاحة الحسابية لا علاقة له بالنفي المنطقي. الخيار B ينكر الحل الموثق بمثال محلول كامل. الخيار C يقترح تعليمة AND بقناع كامل من الواحدات، وهذا يحافظ على القيمة كما هي دون نفيها.

الجواب: د.

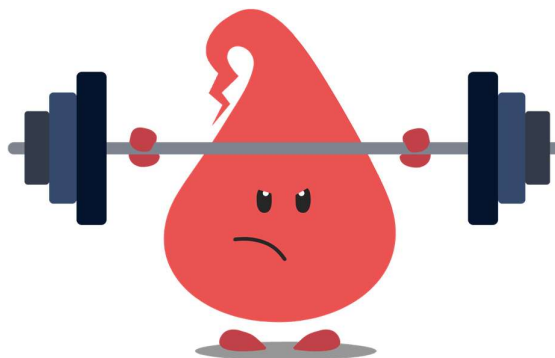
س5: السجل $\$t1$ يحمل القيمة 1001000 (0x90)، ونريد استخراج البتات الأربع العلوية فقط منها وحفظ الناتج في $\$t0$. ما التسلسل الصحيح من التعليمات لتحقيق ذلك؟

- أ) $\text{addi } \$t2, \$zero, 0xF0$ ثم $\$t0, \$t1, \$t2$ and
 ب) $\text{addi } \$t2, \$zero, 0x0F$ ثم $\$t0, \$t1, \$t2$ and
 ج) $\text{addi } \$t2, \$zero, 0xF0$ ثم $\$t0, \$t1, \$t2$ or
 د) $\text{addi } \$t0, \$t0, 0xF0$ ثم $\text{and } \$t0, \$t1, \$zero$

الشرح: القناع $xF00$ (بتات علوية = واحدات، بتات سفلية = أصفار) مع تعليمة AND يعزل البتات الأربع العلوية فقط، مطابقاً للحل المعتمد.

الأخطاء: الخيار B يستخدم القناع $x0F0$ الذي يعزل البتات السفلية بدلاً من العلوية. الخيار C يستخدم OR بدلاً من AND، وهذا لا يعزل أي بتات بل يفعلها جميعاً في مواضع القناع. الخيار D يصفّر $\$t0$ بالكامل أولاً عبر AND مع $\$zero$ ، مما يفقد القيمة الأصلية قبل أي استخراج.

الجواب: أ.



س6: السجل \$t2 يحمل القيمة العشرية 4 (0100 0000)، ونريد تفعيل بت معين ليصبح العدد يساوي 12 عشرياً، مع حفظ الناتج في \$t0. ما التسلسل الصحيح من التعليمات لتحقيق ذلك؟

- (أ) `addi $t1, $zero, 0x08` ثم `$t0, $t2, $t1 and`
 (ب) `addi $t1, $zero, 0x08` ثم `$t0, $t2, $t1 or`
 (ج) `addi $t1, $zero, 0x0C` ثم `$t0, $t2, $t1 and`
 (د) `addi $t0, $t0, 0x08` ثم `$t0, $t2, $zero or`

الشرح: 4 (0100 0000) OR بالقيمة $x080$ (1000 0000) يعطي 12 (1100 0000)، حيث تعليمة OR تُفَعِّل البت المطلوب دون التأثير على باقي البتات، مطابقاً للحل المعتمد.

الأخطاء: الخيار A و B يستخدمان AND بدلاً من OR، ولا يمكنها تفعيل بت كان أصلاً صفراً (تعزل فقط، لا تُفَعِّل). الخيار D يخلط بين خطوة تمرير القيمة الأصلية عبر OR مع \$zero (لا تغيّرها) ثم يحاول تعديلها بتعليمة إضافية غير ضرورية بترتيب مختلف عن الحل المعتمد.

الجواب: ب.

س7: عند قراءة المعالج لأول 6 بتات من تعليمة تفرع شرطي، يتعرف مباشرة على نوعها اعتماداً على قيمة `opcode`. كيف يميّز المعالج بين تعليمتي `bneq` و `beq`؟

- (أ) `beq` قيمة `opcode` لها 5، `bneq` قيمة `opcode` لها 4
 (ب) كلتا التعليمتين تحملان القيمة نفسها (`opcode=4`) ويُميّز بينهما من حقل `funct`
 (ج) `beq` قيمة `opcode` لها 4، `bneq` قيمة `opcode` لها 5
 (د) لا يوجد `opcode` ثابت لهما؛ يعتمد الأمر على قيمة السجلين `rs` و `rt`

الشرح: `opcode` الخاص بـ `beq` يساوي 4، و `opcode` الخاص بـ `bneq` يساوي 5، وبذلك يميّز المعالج بينهما فور قراءة أول 6 بتات من التعليمة.

الأخطاء: الخيار A يعكس القيمتين. الخيار C يخترع اعتماداً على حقل funct غير موجود أصلاً في هذا السياق (فكلتا التعليميتين من نوع type-I، لا type-R). الخيار D ينكر وجود القيمة الثابتة الموثقة صراحة لكل من التعليميتين.

الجواب: ج.

س8: تُعرّف الكتلة الأساسية (Basic Block) بأنها تتابع من التعليمات له خصائص محددة. ما الخصائص التي تحققها الكتلة الأساسية بحسب هذا التعريف؟

- (أ) يجب أن تحتوي على فرع واحد على الأقل في منتصفها لضمان المرونة
- (ب) لا حد أقصى أو أدنى لعدد نقاط الدخول أو الخروج طالما التعليمات متتالية
- (ج) يجب أن تنتهي دائماً بتعليمة jal فقط دون أي نوع آخر من التفرع
- (د) موجه دخول واحد فقط، وموجه خروج واحد فقط، دون أي فروع مغروسة بداخلها عدا في النهاية، ودون أن يكون أي فرع من البرنامج هدفه تعليمة في منتصف الكتلة

الشرح: الكتلة الأساسية لها موجه دخول واحد وموجه خروج واحد، ولا تحتوي على أي فروع مغروسة بداخلها عدا في النهاية، ولا يستهدفها أي فرع من البرنامج في منتصفها (عدا في بدايتها).

الأخطاء: الخيار B يناقض التعريف مباشرة الذي ينص على عدم وجود فروع مغروسة داخل الكتلة عدا في نهايتها. الخيار C ينكر القيد الأساسي (دخول وخروج وحيدين) الذي يحدد الكتلة الأساسية أصلاً. الخيار D يخص نوعاً واحداً فقط من التفرع (jal) دون مبرر، بينما التعريف لا يقيّد نوع تعليمة النهاية بهذا الشكل.

الجواب: د.

استمر فلم يجب إلا القليل ...

س9: لدينا الكود بلغة C: $\text{if}(i==j) f=g+h; \text{else } f=g-h$ ؛ حيث يُقابل المتغيرات f, g, h, i, j التسلسل $s0$ إلى $s4$. ما البنية الصحيحة لترجمة هذا الكود إلى MIPS Assembly؟

- (أ) $\text{bne } \$s3, \$s4, \text{Else}$ ثم $\text{add } \$s0, \$s1, \$s2$ ثم Exit ثم $\text{Else: sub } \$s0, \$s1, \$s2$ ثم Exit
- (ب) $\text{beq } \$s3, \$s4, \text{Else}$ ثم $\text{add } \$s0, \$s1, \$s2$ ثم Exit ثم $\text{Else: sub } \$s0, \$s1, \$s2$ ثم Exit
- (ج) $\text{bne } \$s3, \$s4, \text{Else}$ ثم $\text{sub } \$s0, \$s1, \$s2$ ثم Exit ثم $\text{Else: add } \$s0, \$s1, \$s2$ ثم Exit
- (د) $\text{Else: sub } \$s0, \$s1, \$s2$ ثم $\text{add } \$s0, \$s1, \$s2$ ثم $\text{bne } \$s3, \$s4, \text{Exit}$ ثم Exit

الشرح: $\text{bne } \$s3, \$s4, \text{Else}$ يتحقق من عدم تساوي i (أي حالة else)، فإذا لم يتساويا يقفز مباشرة إلى else (sub) ؛ وإن تساويا يكمل تنفيذ add ثم يقفز فوق قسم else بتعليمية Exit .

الأخطاء: الخيار A يستخدم beq بدلاً من bne ، مما يعكس منطق القفز بالكامل. الخيار C يبدّل ترتيب عمليتي الجمع والطرح بين قسمي if و else . الخيار D يقدم قفزة غير منطقية (Else j) قبل فحص الشرط أصلاً، مما يجعل التفرع الشرطي عديم الفائدة.

الجواب: أ.

س10: لدينا الحلقة: $\{ i+=1; \text{while}(\text{save}[i]==k) \}$ ، حيث $s3=i$ ، $s5=k$ ، $s6=\text{عنوان المصفوفة save}$. ما التسلسل الصحيح من MIPS Assembly لهذه الحلقة؟

- (أ) $\text{Loop: add } \$t1, \$s3, \$s6$ ثم $\text{lw } \$t0, 0(\$t1)$ ثم $\text{bne } \$t0, \$s5, \text{Exit}$ ثم $\text{addi } \$s3, \$s3, 1$ ثم Loop ثم Exit
- (ب) $\text{Loop: sll } \$t1, \$s3, 2$ ثم $\text{add } \$t1, \$t1, \$s6$ ثم $\text{lw } \$t0, 0(\$t1)$ ثم $\text{bne } \$t0, \$s5, \text{Exit}$ ثم $\text{addi } \$s3, \$s3, 1$ ثم Loop ثم Exit
- (ج) $\text{Loop: sll } \$t1, \$s3, 2$ ثم $\text{add } \$t1, \$t1, \$s6$ ثم $\text{lw } \$t0, 0(\$t1)$ ثم $\text{beq } \$t0, \$s5, \text{Exit}$ ثم $\text{addi } \$s3, \$s3, 1$ ثم Loop ثم Exit
- (د) $\text{Loop: sll } \$t1, \$s3, 2$ ثم $\text{add } \$t1, \$t1, \$s6$ ثم $\text{lw } \$t0, 0(\$t1)$ ثم $\text{bne } \$t0, \$s5, \text{Exit}$ ثم $\text{addi } \$s3, \$s3, 1$ ثم Loop ثم Exit

الشرح: $sl\ t1, \$s3, 2$ يضرب الفهرس i بـ 4 (لأن كل عنصر في المصفوفة كلمة كاملة 4 بايت)، ثم add يحسب عنوان $save[i]$ ، ثم lw يجلب قيمته، ثم bne تخرج من الحلقة إذا لم يعد $save[i] = k$ ، ثم $addi$ تزيد i ، ثم $loop$ ز تعيد التكرار.

الأخطاء: الخيار B يحذف خطوة $sl\ t$ الضرورية لضرب الفهرس بـ 4 قبل الجمع مع عنوان القاعدة، مما ينتج عنواناً خاطئاً. الخيار C يستخدم beq بدلاً من bne كشرط الخروج، مما يعكس منطق استمرار الحلقة بالكامل. الخيار D يبدّل ترتيب تعليمتي $loop$ و $addi$ ، مما يجعل الحلقة تكرر التنفيذ دون زيادة i أبداً.

الجواب: ب.

س11: تُستخدم تعليمة (set if less than) slt بدلاً من الانتقال المباشر إلى مكان تعليمات أخرى كما تفعل beq و $bneq$. ما الذي تفعله $slt\ rd, rs, rt$ تحديداً؟

(أ) تقفز مباشرة إلى العنوان الموجود في rd إذا تحقق الشرط $rs < rt$

(ب) تبادّل قيمتي rs و rt إذا كان rs أكبر من rt

(ج) تحفظ القيمة 1 في rd إذا كان $rs < rt$ ، وتحفظ 0 خلاف ذلك، دون أي قفز

(د) تطرح rt من rs وتحفظ الناتج الحسابي الكامل في rd

الشرح: إذا كان $rt < rs$ يحفظ Compiler القيمة 1 في rd ، وإلا يحفظ 0، دون أي انتقال أو قفز، بعكس beq و $bneq$.

الأخطاء: الخيار B يخلط بين وظيفة slt (مقارنة وحفظ نتيجة منطقية) ووظيفة تعليمات القفز الشرطي. الخيار C يخترع عملية تبادل غير موجودة في دلالة slt . الخيار D يصف عملية طرح حسابية كاملة بدلاً من نتيجة مقارنة منطقية (0 أو 1).

الجواب: ج.

س12: يتساءل طالب: لماذا لا توفر MIPS تعليمتي bge (branch if greater or g) و blt (branch if less than) $equal$ مباشرة، رغم أنهما كانتا ستبسطان كتابة الحلقات الشرطية؟ ما السبب؟

- (أ) لأن MIPS لا تدعم المقارنات المنطقية إطلاقاً بأي شكل من الأشكال
 (ب) لأن هاتين التعليمتين موجودتان فعلياً في MIPS تحت اسمين مختلفين فقط
 (ج) لأن إضافتهما تتطلب تغيير عدد سجلات المعالج بالكامل
 (د) لأن هاتين التعليمتين معقدتان جداً وتحتاجان دورات ساعة أطول لتنفيذهما، مما يجعل زمن التنفيذ أطول، لذلك تُبنى المقارنات المنطقية من slt و $slti$ و beq و $bneg$ بدلاً منهما

الشرح: التعليمتان bge و blt معقدتان جداً وتحتاجان دورات ساعة أطول، ولذلك تستخدم مجمعات MIPS تعليمات bne و beq و $slti$ و slt مع قيم ثابتة من الصفر ($\$zero$) للحصول على كل أنواع المقارنات المنطقية.

الأخطاء: الخيار B ينكر وجود آلية كاملة لبناء المقارنات المنطقية موضحة صراحة عبر مجموعة التعليمات المتوفرة. الخيار C يخترع ادعاءً غير صحيح؛ فالتعليمتان غير موجودتين إطلاقاً وليس لهما اسم بديل. الخيار D يقحم علاقة غير مذكورة بعدد السجلات في هذا السياق.

الجواب: د.

س13: لدينا السجلان: $\$s0 = 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111$ ، و $\$s1 = 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000$ ، وننفذ: $slt\ \$t0,\$s0,\$s1$ (مقارنة موقعة) ثم $sltu\ \$t1,\$s0,\$s1$ (مقارنة غير موقعة). ما القيمتان الناتجتان في $\$t0$ و $\$t1$ ؟

- (أ) $t0 = 1$ (لأن $-1 > 1+$ عند تفسيرها كمقارنة غير موقعة)
 (ب) $t0 = 0$ و $t1 = 0$ في كلتا الحالتين
 (ج) $t0 = 1$ و $t1 = 1$ في كلتا الحالتين
 (د) $t0 = 0$ (لأن $-1 < 1+$ دائماً)، و $t1 = 1$ (لأن $1 > 4,294,967,295$ عند التفسير غير الموقّع)

الشرح: عند تفسير $s0$ كعدد موقع (Two's Complement) فإنه يمثل -1 ، وبما أن $1 > -1$ فإن slt تعطي $t0=1$. أما عند تفسيره كعدد غير موقع فإنه يمثل $4,294,967,295$ ، وبما أنه أكبر من 1 فإن $sltu$ تعطي $t1=0$.

الأخطاء: الخيار B وC يطبقان نتيجة واحدة ثابتة على كلا التعليمتين، متجاهلين أن التفسير المختلف للسجل نفسه (موقع/غير موقع) يغير نتيجة المقارنة تماماً كما يوضح المثال. الخيار D يعكس النتيجة الصحيحة تماماً.

الجواب: أ.

س14: للتحقق من أن عنصراً x يقع ضمن حدود مصفوفة حجمها y (أي لاكتشاف الوصول خارج المصفوفة، سواء كان x سالباً أو أكبر من أو يساوي y)، يُستخدم كود يعتمد على تعليمة $sltu$ ثم beq . لماذا تُستخدم تعليمة $sltu$ تحديداً (وليس slt) في هذا الفحص؟

- أ) لأن $sltu$ أسرع من slt في التنفيذ من ناحية عدد دورات الساعة
- ب) لأن استخدام $sltu$ (المقارنة غير الموقعة) يجعل أي قيمة سالبة لـ x تُفسر كعدد موجب كبير جداً يتجاوز حدود المصفوفة تلقائياً، فيكتشف الفحص الواحد كلتا الحالتين الخاطئتين (x سالب أو x أكبر من y) في مقارنة واحدة
- ج) لأن slt غير موجودة أصلاً في نوع تعليمات type-R
- د) لأن $sltu$ هي التعليمة الوحيدة القادرة على المقارنة مع السجل $\$zero$

الشرح: بما أن العناصر خارج المصفوفة تكون إما سالبة أو أكبر من/تساوي حجم المصفوفة، فإن تفسير x كعدد غير موقع ($sltu$) يحول أي قيمة سالبة إلى عدد ضخم يتجاوز y تلقائياً، فتكفي مقارنة واحدة ($x < y$) بالتفسير غير الموقع لاكتشاف كلا نوعي الخطأ معاً.

الأخطاء: الخيار B يخترع فرق سرعة غير مذكور في هذا السياق. الخيار C خاطئ فعلياً؛ slt موجودة وتُستخدم في سياقات أخرى من نوع type-R. الخيار D يخترع قيداً غير موجود؛ فكلا التعليمتين slt و $sltu$ يمكن استخدامهما مع أي سجل بما فيه $\$zero$.

الجواب: ب.

س15: عند استدعاء أي دالة (إجراء)، يمر المعالج بست خطوات متتالية. ما هذه الخطوات الست بالترتيب؟

- (أ) التنفيذ، تمرير المعاملات، الرجوع، نقل التحكم، وضع قيمة العودة، حجز مساحة
- (ب) نقل التحكم، التنفيذ، حجز مساحة، تمرير المعاملات، الرجوع، وضع قيمة العودة
- (ج) تمرير المعاملات، نقل التحكم، حجز مساحة للمتغيرات المحلية، التنفيذ، وضع قيمة العودة، الرجوع لنقطة الاستدعاء
- (د) حجز مساحة، الرجوع، التنفيذ، نقل التحكم، تمرير المعاملات، وضع قيمة العودة

الشرح: الخطوات الست بالترتيب الصحيح: تمرير المعاملات (وضعها في سجلات خاصة) ← نقل التحكم (القفز لعنوان الدالة) ← حجز مساحة في الـ stack للمتغيرات المحلية ← التنفيذ ← وضع قيمة العودة في سجل خاص ← الرجوع لنقطة الاستدعاء.

الأخطاء: الخيارات B و C و D تعرض الخطوات الست الصحيحة نفسها لكن بترتيب مبعثر لا يطابق التسلسل الفعلي الذي يمر به المعالج.

الجواب: ج.

س16: التعليمة (jump and link) jal procedureAddress تُستخدم عند استدعاء إجراء. ما الوظيفة الدقيقة

لهذه التعليمة؟

- (أ) تنسخ القيمة الموجودة في $ra\$$ إلى PC مباشرة دون أي حفظ
- (ب) تحجز مساحة في المكس دون أي قفز أو حفظ عنوان
- (ج) تضع عنوان الإجراء الحالي (وليس التالي) في $ra\$$ ، ثم تكمل التنفيذ التسلسلي دون قفز
- (د) تضع عنوان التعليمة التالية ($PC+4$) في سجل $ra\$$ ، ثم تقفز إلى عنوان الهدف (PC ← عنوان الإجراء)

الشرح: jal تضع عنوان التعليمة التالية (بعد إضافة 4 لمكان توقف PC) في سجل $ra\$$ (سجل عنوان العودة)، ثم تقفز إلى عنوان الهدف عبر تحديث PC ← عنوان الهدف.

الأخطاء: الخيار B يصف وظيفة jr وليس jal. الخيار C ينكر الوظيفتين الأساسيتين لـ jal (الحفظ والقفز) معاً. الخيار D يصف عنواناً خاطئاً (الحالي بدل التالي) وينكر خطوة القفز الفعلية.

الجواب: د.

س17: تُكتب التعليمه ra\$ jr في آخر سطر داخل أي دالة. ما وظيفتها تحديداً؟

- (أ) تنسخ القيمة الموجودة في ra\$ إلى PC
 (ب) تحفظ عنوان جديد في ra\$ قبل إنهاء الدالة
 (ج) تحجز مساحة إضافية في المكس قبل الخروج من الدالة
 (د) تقفز إلى عنوان ثابت مكتوب مباشرة داخل التعليمه نفسها

الشرح: ra\$ jr تنسخ القيمة الموجودة في ra\$ إلى PC، فيعود المعالج بأمان إلى نقطة الاستدعاء في البرنامج الرئيسي.

الأخطاء: الخيار B يعكس اتجاه النسخ الصحيح (من ra\$ إلى PC، لا العكس). الخيار C يقحم وظيفة غير متعلقة برز إطلاقاً. الخيار D يصف سلوك تعليمه قفز مباشر (ك ز) وليس القفز غير المباشر عبر سجل الذي تقوم به jr.

الجواب: أ.

س18: خُصّ السجل رقم 29 ليكون مؤشر المكس (Stack Pointer)، ويحفظ عنوان قمة المكس. في أي اتجاه ينمو المكس بحسب المحاضرة؟

- (أ) ينمو من العناوين الدنيا صعوداً إلى العناوين العليا
 (ب) ينمو من العناوين العليا نزولاً إلى العناوين الدنيا
 (ج) لا ينمو في أي اتجاه؛ حجمه ثابت طوال تنفيذ البرنامج
 (د) ينمو في كلا الاتجاهين بالتساوي حسب الحاجة



الشرح: ينمو المكس من العناوين العليا نزولاً إلى العناوين الدنيا، وتحديداً بدءاً من العنوان 7c7fffff.

الأخطاء: الخيار B يعكس اتجاه النمو الصحيح. الخيار C ينكر الطبيعة الديناميكية للمكس أثناء استدعاءات الإجراءات المتكررة. الخيار D يخترع سلوكاً غير موصوف؛ فالمكس ينمو باتجاه واحد محدد فقط.

الجواب: ب.

س19: عند تنفيذ إجراء يحتاج لاستخدام سجل محفوظ مثل \$0؛ دون فقدان قيمته السابقة، يُحفظ في المكس أولاً ثم يُستعاد لاحقاً. ما الترتيب الصحيح للتعليمات المستخدمة (حز المساحة، الحفظ، الاستعادة، إعادة المساحة)؟

- أ) (sw \$s0,0(\$sp (حفظ) ثم addi \$sp,\$sp,-4 (حز مساحة) ... ثم addi \$sp,\$sp,4 (إعادة المساحة) ثم (lw \$s0,0(\$sp (استعادة)
- ب) addi \$sp,\$sp,4 (حز مساحة) ثم (sw \$s0,0(\$sp (حفظ) ... ثم (lw \$s0,0(\$sp (استعادة) ثم addi \$sp,\$sp,-4 (إعادة المساحة)
- ج) addi \$sp,\$sp,-4 (حز مساحة) ثم (sw \$s0,0(\$sp (حفظ) ... ثم (lw \$s0,0(\$sp (استعادة) ثم addi \$sp,\$sp,4 (إعادة المساحة)
- د) (lw \$s0,0(\$sp (استعادة أولاً) ثم addi \$sp,\$sp,-4 ... ثم (sw \$s0,0(\$sp ثم addi \$sp,\$sp,4

الشرح: الترتيب الصحيح: addi \$sp,\$sp,-4 (طرح 4 من مؤشر المكس لتفريغ مساحة) ثم sw \$s0,0(\$sp (حفظ القيمة)، وبعد انتهاء الإجراء: lw \$s0,0(\$sp (استعادة القيمة) ثم addi \$sp,\$sp,4 (إعادة مؤشر المكس لرد المساحة).

الأخطاء: الخيار B يبدّل ترتيب خطوتي الحز والحفظ، وكذلك ترتيب خطوتي الاستعادة وإعادة المساحة. الخيار C يعكس إشارتي ال4 (يستخدم +4 للحز و-4 لإعادة المساحة، بينما الصحيح عكس ذلك تماماً بما يتوافق مع نمو المكس نزولاً). الخيار D يستعيد القيمة قبل حتى حز المساحة أو حفظها، وهذا غير منطقي تماماً.

الجواب: ج.

س20: الذاكرة في برامج MIPS مقسمة إلى أقسام محددة: النص (Text)، البيانات الثابتة (Static Data)، البيانات الديناميكية (Heap)، المكس (Stack)، والجزء المحجوز (Reserved). أي وصف يطابق دور وموقع كل قسم بدقة؟

- أ) المكس يخزن كود البرنامج، والنص يُستخدم لحفظ المتغيرات المحلية فقط
- ب) الheap والمكس كلاهما ينموان صعوداً باتجاه بعضهما البعض دون أي فارق بينهما
- ج) الجزء المحجوز مخصص لتخزين المتغيرات العامة (Static)، بينما القسم الثابت مخصص لنظام التشغيل
- د) النص يخزن كود البرنامج ويبدأ من عنوان منخفض، والمكس يبدأ من أعلى الذاكرة وينمو نزولاً باتجاه الheap، بينما الheap ينمو صعوداً على عكس المكس

الشرح: النص (Text) يخزن التعليمات ويبدأ من عنوان منخفض نسبياً؛ البيانات الثابتة تُصل إليها عبر `op$`؛ `Heap` ينمو للأعلى؛ المكسد يبدأ من أعلى الذاكرة وينمو نزولاً باتجاه `heap`؛ والجزء المحجوز مخصص لنظام التشغيل والمقاطعات والإدخال/الإخراج.

الأخطاء: الخيار B يعكس أدوار قسمي النص والمكدس تماماً. الخيار C ينكر الفرق الموثق في اتجاه نمو كل من `heap` والمكدس (أحدهما يصعد والآخر ينزل نحو بعضهما). الخيار D يعكس أدوار قسمي البيانات الثابتة والجزء المحجوز.

الجواب: د.

س21: تعليمات التفرع الشرطي في MIPS من نوع I-type، وتخصص 16 بت فقط لتمثيل العنوان الهدف، بينما عناوين الذاكرة في MIPS حجمها 32 بت. كيف يُحسب العنوان الفعلي للقفز عند تطابق الشرط، وما نقطة البداية المرجعية لهذا الحساب؟

أ) العنوان $(PC+4) + (offset \times 4)$ ، حيث تكون نقطة البداية المرجعية هي عنوان التعليمة التالية للتفرع مباشرة $(PC+4)$ ، وليس عنوان تعليمة الشرط نفسها

ب) العنوان $PC + offset$ مباشرة دون ضرب بـ 4، ونقطة البداية هي تعليمة الشرط نفسها

ج) العنوان $(PC-4) + (offset \times 4)$ ، ونقطة البداية هي تعليمة الشرط نفسها

د) العنوان $offset \times 4$ فقط، دون أي علاقة بموقع `PC` الحالي إطلاقاً

الشرح: المعالج لا يحفظ العنوان الحقيقي في الـ 16 بت بل يحفظ المسافة (`offset`) بالنسبة لمكانه الحالي، والنقطة المرجعية للحساب هي التعليمة التي تلي تعليمة الشرط مباشرة $(PC+4)$ ، وليس تعليمة الشرط نفسها؛ وتُضرب المسافة بـ 4 لأن كل تعليمة تشغل 4 بايت (`word-aligning`).

الأخطاء: الخيار B ينسى الضرب بـ 4 الضروري لتحويل عدد الكلمات إلى عدد بايتات فعلي. الخيار C يستخدم $PC-4$ بدلاً من $PC+4$ كنقطة مرجعية، عكس ما هو موثق صراحة. الخيار D يتجاهل نقطة البداية المرجعية (`PC`) بالكامل، رغم أنها جزء أساسي من معادلة الحساب.

الجواب: أ.

س22: تعليمات القفز غير المشروط مثل ز واز تعتمد على تنسيق I-type، حيث يُرمز العنوان الكامل بـ26 بت فقط رغم أن عناوين MIPS الفعلية 32 بت. ما الخطوتان المستخدمتان لتحويل هذه الـ26 بت إلى عنوان هدف كامل بـ32 بت (Pseudo-Direct Addressing)؟

- (أ) إزاحة الـ26 بت لليمين بخانتين لتصبح 24 بت، ثم إضافة صفرين في أسفل العنوان مباشرة
- (ب) إزاحة الـ26 بت لليسار بخانتين (ضربها بـ4) لتصبح 28 بت، ثم دمج الخانات الأربع العليا من عداد البرنامج الحالي (PC) في أعلى العنوان الناتج
- (ج) دمج الـ26 بت مباشرة مع كامل قيمة (32) PC بت دون أي إزاحة مسبقة
- (د) تحويل الـ26 بت إلى مكملها الثاني (Two's Complement) ثم إضافتها مباشرة إلى PC

الشرح: الخطوة الأولى: word-aligning، أي إزاحة الـ26 بت لليسار بخانتين (ضرب بـ4) لتصبح 28 بت. الخطوة الثانية: أخذ الخانات العليا الأربع من PC الحالي ودمجها في أعلى العنوان الناتج، بحيث تكتمل الـ32 بت النهائية.

الأخطاء: الخيار B يعكس اتجاه الإزاحة (يمين بدلاً من يسار) ويصف نتيجة خاطئة (24 بت بدلاً من 28). الخيار C يتجاهل خطوة الإزاحة الأساسية (word-aligning) تماماً. الخيار D يخترع عملية تحويل (Two's Complement) غير مذكورة إطلاقاً في آلية العنونة شبه المباشرة.

الجواب: ب.

س23: يريد مبرمج تخزين ثابت كامل حجمه 32 بت في السجل \$O، علماً أن حقل Immediate في تعليمة واحدة يتسع لـ16 بت فقط. ما التسلسل الصحيح من التعليمات لتحقيق ذلك، إذا كان النصف العلوي من الثابت يساوي 61 والنصف السفلي يساوي 2304؟

- (أ) `ori $s0, $zero, 61` ثم `lui $s0, 2304`
- (ب) `lui $s0, 2304` ثم `ori $s0, $s0, 61`
- (ج) `lui $s0, 61` ثم `ori $s0, $s0, 2304`
- (د) `addi $s0, $zero, 61` ثم `addi $s0, $s0, 2304`

الشرح: تعليمة lui تضع النصف العلوي من الثابت (61) في النصف العلوي من السجل وتصفّر النصف السفلي، ثم تعليمة ori تدمج النصف السفلي (2304) مع القيمة الموجودة، لتكتمل القيمة الكاملة بـ 32 بت.

الأخطاء: الخيار B يبدّل ترتيب التعليمتين ويستخدم ori بدلاً من lui للنصف العلوي، وهذا لا يضع القيمة في الموضع الصحيح داخل السجل. الخيار C يبدّل قيمتي النصفين العلوي والسفلي عن موضعهما الصحيحين. الخيار D يستخدم addi مرتين، وهذا لا يضع النصف الأول في الموضع العلوي الصحيح من السجل كما تفعل lui تحديداً.

الجواب: ج.

س24: توجد خمسة طرق (أنماط) للوصول للبيانات في MIPS بحسب ملخص أنماط العنونة. ما هذه الأنماط الخمسة وتعريف كل منها؟

(أ) عنونة فورية فقط، وعنونة سجل فقط، مكررتين أربع مرات لتشكيل الأنماط الخمسة

(ب) عنونة القفز، عنونة التفرع، عنونة الحفظ، عنونة الاسترجاع، وعنونة المقارنة

(ج) أربعة أنماط فقط لا خمسة: فورية، سجل، قاعدية، و PC-relative دون Pseudodirect

(د) العنونة الفورية (immediate: المعامل ثابت داخل التعليمة)، عنونة السجل (Register: المعامل في السجل)، العنونة القاعدية/الإزاحة (Base/Displacement: عنوان الذاكرة = قيمة سجل + ثابت)، العنونة المرتبطة بعدد البرنامج (PC-relative: للتفرع)، والعنونة شبه المباشرة (Pseudodirect: لتعليمات القفز j و jal)

الشرح: الأنماط الخمسة: العنونة الفورية، عنونة السجل، العنونة القاعدية/الإزاحة، العنونة المرتبطة بعدد البرنامج (للتفرع)، والعنونة شبه المباشرة (لتعليمات القفز).

الأخطاء: الخيار B يخترع تكراراً غير منطقي لنمطين فقط بدل خمسة أنماط متميزة. الخيار C يخترع تصنيفاً كاملاً غير مطابق للأنماط الخمسة الفعلية الموثقة. الخيار D يحذف نمط العنونة شبه المباشرة رغم ذكره صراحة كخامس الأنماط.

الجواب: د.

س25: في دالة strcpy المكتوبة بلغة Assembly، يُستخدم العداد i لحساب عنوان كل محرف (char) ضمن السلسلتين النصيتين x وy. لماذا لا يُضرب العداد i ب4 عند حساب هذه العناوين، بعكس ما يحدث عادة عند التعامل مع عناصر مصفوفة من نوع كلمة (Word)؟

أ) لأن حجم المحرف (char) في الذاكرة هو بايت واحد فقط، وليس كلمة كاملة (4 بايت)، فلا حاجة لضرب الفهرس ب4 كما يحدث مع مصفوفات الكلمات

ب) لأن دالة strcpy لا تتعامل مع عناوين ذاكرة فعلية إطلاقاً

ج) لأن الضرب ب4 يحدث تلقائياً داخل تعليمتي lbu وsbu دون كتابته صراحة في الكود

د) لأن المحارف في MIPS تُعْمَل دائماً بأربعة بايتات وليس بايتاً واحداً

الشرح: حجم المحرف char في الذاكرة هو بايت واحد فقط، بعكس عناصر مصفوفة من نوع Integer (4 بايت)، ولذلك لا يُضرب العداد i ب4 عند حساب عناوين المحارف في هذه الدالة تحديداً.

الأخطاء: الخيار B ينكر استخدام عناوين ذاكرة فعلية رغم أن الكود بأكمله يعتمد على lbu وsbu للوصول لعناوين محددة. الخيار C يخترع سلوكاً ضمناً غير موجود؛ فلا يوجد ضرب تلقائي مخفي داخل هاتين التعليمتين. الخيار D يناقض الحقيقة الأساسية التي يقوم عليها التفسير نفسه (أن المحرف بايت واحد لا أربعة).

الجواب: أ.

IT IS DONE!